



แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอน
และงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ

นายรัฐพิงษ์ ตะเพียนทอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ

นายฐิติพงษ์ ตะเพียนทอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Cluster-Based Platform for Supporting Teaching and Academic Activities
in the Department of Information Technology

MR.THITIPONG TAPIANTHONG

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE BACHELOR'S DEGREE OF ENGINEERING
PROGRAM IN INFORMATION AND NETWORK ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY
FACULTY OF INDUSTRIAL TECHNOLOGY AND MANAGEMENT
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
ACADEMIC YEAR 2025
COPYRIGHT OF KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK



ใบรับรองปริญญาานิพนธ์
คณะเทคโนโลยีและการจัดการอุตสาหกรรม
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เรื่อง แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยี
สารสนเทศ
โดย นายฐิติพงษ์ ตะเพียนทอง

ได้รับอนุมัติให้นับเป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย

_____ คนบดี
(ผู้ช่วยศาสตราจารย์ ดร.กฤษฎากร บุตดาจันทร์)

คณะกรรมการสอบโครงการสหกิจศึกษา

_____ ประธานกรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ศรายุทธ ธเนศสกุลวัฒนา)

_____ กรรมการ
(อาจารย์ ดร.วัชรชัย คงศิริวัฒนา)

_____ กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ)

ชื่อ : นายฐิติพงษ์ ตะเพียนทอง
ชื่อปริญญาบัตร : แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงาน
วิชาการภาควิชาเทคโนโลยีสารสนเทศ
สาขาวิชา : วิศวกรรมสารสนเทศและเครือข่าย
: มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
อาจารย์ที่ปรึกษาปริญญาบัตร : ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ
ปีการศึกษา : 2568

บทคัดย่อ

ปริญญาบัตร "แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ" ฉบับนี้ มีวัตถุประสงค์เพื่อพัฒนาแพลตฟอร์มคลัสเตอร์เสมือนที่สามารถสร้างและจัดการเครื่องเสมือนลินุกซ์ได้อย่างรวดเร็วและมีประสิทธิภาพ เพื่อสนับสนุนการเรียนการสอนและการทำโครงการในภาควิชาเทคโนโลยีสารสนเทศ และพัฒนาเว็บแอปพลิเคชันแบบ Role-Based Access Control (RBAC) ที่รองรับการใช้งานของทั้งนักศึกษาและอาจารย์ โดยคำนึงถึงความสะดวกในการใช้งาน เสถียรภาพของระบบ และความปลอดภัยในการเข้าถึง ระบบประกอบด้วยสามส่วนหลัก ได้แก่ ส่วนติดต่อผู้ใช้งาน บริการ API สำหรับจัดการการยืนยันตัวตน การจัดการผู้ใช้ และเชื่อมต่อระหว่างส่วนติดต่อผู้ใช้และบริการจัดการเครื่องเสมือน และบริการจัดการเครื่องเสมือนที่เชื่อมต่อกับระบบ Hypervisor เพื่อสร้าง กำหนดค่า และจัดการเครื่องเสมือน ระบบสามารถลดความซับซ้อนในการตั้งค่าสภาพแวดล้อมสำหรับการใช้งานระบบลินุกซ์ เพิ่มประสิทธิภาพการใช้ทรัพยากรเซิร์ฟเวอร์ และสนับสนุนการเรียนการสอนที่เกี่ยวข้องกับระบบเซิร์ฟเวอร์ การพัฒนาแอปพลิเคชัน และการจัดการโครงสร้างพื้นฐานด้านไอทีได้อย่างมีประสิทธิภาพ

(ปริญญาบัตรมีจำนวนทั้งสิ้น 58 หน้า)

Name : Mr.Thitipong Taphanthong
Project Title : Cluster-Based Platform for Supporting Teaching and Academic Activities in the Department of Information Technology
Major Field : Information and Network Engineering
: King Mongkut's University of Technology North Bangkok
Project Advisor : Asst. Prof. Dr.Khanista Namee
Academic Year : 2025

Abstract

This project, "Cluster-Based Platform for Supporting Teaching and Academic Activities in the Department of Information Technology," aims to develop a virtual cluster platform capable of rapidly and efficiently creating and managing Linux virtual machines to support teaching and project work in the Department of Information Technology. The project also focuses on developing a Role-Based Access Control (RBAC) web application that supports both students and faculty, emphasizing ease of use, system stability, and access security. The system consists of three main components: a user interface, an API service for handling authentication and user management and communication between the frontend and VM operations service, and a VM management service that interfaces with the Hypervisor system to create, configure, and manage virtual machines. The system reduces the complexity of setting up Linux environments, improves server resource utilization efficiency, and effectively supports teaching related to server systems, application development, and IT infrastructure management.

(Total 58 pages)

Project Advisor

กิตติกรรมประกาศ

ปริญญานิพนธ์ "แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาค
วิชาเทคโนโลยีสารสนเทศ" จะไม่สามารถสำเร็จลุล่วงไปได้ด้วยดีหากไม่ได้รับการช่วยเหลือและความ
อนุเคราะห์จากบุคคลหลายท่าน ทางผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ ซึ่งเป็น
อาจารย์ที่ปรึกษา ที่ได้กรุณาให้คำปรึกษาแนะนำ และให้แนวทางในการพัฒนาแพลตฟอร์มคลัสเตอร์และ
การจัดทำปริญญานิพนธ์ รวมทั้งคณาจารย์ภาควิชาเทคโนโลยีสารสนเทศที่ได้ให้ความรู้ด้านเทคโนโลยี
และการพัฒนาระบบให้กับผู้จัดทำเพื่อนำมาประยุกต์ในการพัฒนาโครงการนี้ อีกทั้งขอขอบคุณภาควิชา
เทคโนโลยีสารสนเทศที่ให้การสนับสนุนโครงสร้างพื้นฐานและทรัพยากรที่จำเป็นสำหรับการพัฒนาและ
ทดสอบระบบ

ท้ายที่สุดนี้ ทางผู้จัดทำขอน้อมรำลึกพระคุณบิดา มารดา ผู้ซึ่งมีพระคุณอย่างสูงสุดที่ให้ความ
อุปการะผู้จัดทำมาโดยตลอด รวมทั้งผู้มีพระคุณทุกท่าน คณะผู้จัดทำรู้สึกซาบซึ้งเป็นอย่างยิ่ง จึงใคร่ขอ
ขอบพระคุณเป็นอย่างสูงมา ณ โอกาสนี้ด้วย

ฐิติพงษ์ ตะเพียนทอง

สารบัญ

หน้า

บทคัดย่อภาษาไทย	ก
บทคัดย่อภาษาอังกฤษ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญภาพ	ฉ
สารบัญตาราง	ช
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญ	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของการทำโครงการ	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 แนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้อง	5
2.1 แนวคิดพื้นฐานเกี่ยวกับโครงการ	5
2.2 แนวคิดและทฤษฎีที่เกี่ยวข้องกับแพลตฟอร์มคลัสเตอร์เสมือน	7
2.3 รายงานการค้นคว้า การศึกษา หรืองานวิจัยที่เกี่ยวข้อง	10
บทที่ 3 วิธีการดำเนินงานและการออกแบบระบบ	13
3.1 การออกแบบสถาปัตยกรรมระบบ	13
3.2 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา	30
บทที่ 4 ผลการดำเนินงาน	32
4.1 โครงสร้างโปรเจกต์หลัก	32
4.2 โครงสร้างแอปพลิเคชันและบริการ	33
4.3 สถาปัตยกรรมระบบและเทคโนโลยี	34
4.4 เอกสารประกอบการใช้งาน API และส่วนติดต่อผู้ใช้	35
4.5 ตัวอย่างการใช้งานระบบตามบทบาทผู้ใช้	38
4.6 ผลการทดสอบประสิทธิภาพระบบ	44
บทที่ 5 สรุปผลการดำเนินงาน	48
5.1 สรุปผลการดำเนินงาน	48
5.2 ปัญหาและอุปสรรค	48
5.3 การแก้ปัญหา	49
5.4 ข้อเสนอแนะ	49
บรรณานุกรม	50

สารบัญ (ต่อ)

	หน้า
ภาคผนวก	52
ภาคผนวก ก: การติดตั้งและใช้งาน Cloud-Based Platform	53
ก.1 การติดตั้งและใช้งาน Cloud-Based Platform	54
ก.2 การเริ่มต้นใช้งานระบบ	54
ก.3 การใช้งานส่วนประกอบต่างๆ ของระบบ	55
ก.4 การตรวจสอบและติดตามระบบ	56
ก.5 การแก้ไขปัญหาเบื้องต้น	57

สารบัญภาพ

ภาพที่		หน้า
3-1	แผนภาพการทำงานและลำดับขั้นตอนโดยรวมของโปรแกรม	13
3-2	แผนภาพบริบทของระบบแพลตฟอร์มคลัสเตอร์เสมือน	14
3-3	แผนภาพการไหลของข้อมูลระดับ 0	15
3-4	แผนภาพการไหลของข้อมูลระดับที่ 1 (Frontend)	16
3-5	แผนภาพการไหลของข้อมูลระดับที่ 1 (Backend)	16
3-6	แผนภาพการไหลของข้อมูลระดับที่ 1 (VM Management)	17
3-7	หน้าจอเข้าสู่ระบบ (Login Page)	18
3-8	หน้าจอแดชบอร์ด (Dashboard)	19
3-9	หน้าจอจัดการเครื่องเสมือน (Instance Management)	19
3-10	หน้าจอคำร้องขอเครื่องเสมือน (Instance Request)	20
3-11	หน้าจอจัดการคำร้องขอ (Request Management)	20
3-12	แผนภาพโครงสร้างฐานข้อมูลของระบบ (Logical Schema)	22
4-1	โครงสร้างโปรเจกต์ทั้งหมด	34
4-2	สถาปัตยกรรมระบบโดยรวม	35
4-3	หน้าเอกสาร API Documentation	36
4-4	หน้าแรกของระบบ (Landing Page)	37
4-5	Dashboard หลักสำหรับนักศึกษา	39
4-6	หน้าจัดการ VM Instances สำหรับนักศึกษา	39
4-7	หน้าจอขอสร้าง VM Instance ใหม่สำหรับนักศึกษา	40
4-8	Dashboard หลักสำหรับอาจารย์และเจ้าหน้าที่	41
4-9	หน้าจัดการหลักสูตรและรายวิชา	42
4-10	หน้าจัดการเทอมการศึกษา	42
4-11	หน้าจัดการรายชื่ออาจารย์	43
4-12	หน้าอนุมัติคำขอที่รอตําเนินการ	43
ก-1	คำสั่งสำหรับโคลนโปรเจกต์	54
ก-2	คำสั่งสำหรับติดตั้ง Dependencies	54
ก-3	การตั้งค่า Environment Variables	55
ก-4	การเริ่มต้น Infrastructure Services	55
ก-5	การเริ่มต้น Midori Frontend	55
ก-6	การเริ่มต้น Momoi API Service	56

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
ก-7	การเริ่มต้น Yuzu VM Operations Service
	56

สารบัญตาราง

ตารางที่		หน้า
3-1	สรุปตาราง คีย์หลัก และคีย์ต่างประเทศจาก Prisma Schema	23
3-2	รายการ Enum และค่าที่อนุญาตในสคีมาฐานข้อมูล	23
3-3	รายละเอียดฟิลด์ของตาราง user	24
3-4	รายละเอียดฟิลด์ของตาราง session	24
3-5	รายละเอียดฟิลด์ของตาราง account	25
3-6	รายละเอียดฟิลด์ของตาราง verification	25
3-7	รายละเอียดฟิลด์ของตาราง staff_list	25
3-8	รายละเอียดฟิลด์ของตาราง user_public_key	26
3-9	รายละเอียดฟิลด์ของตาราง network	26
3-10	รายละเอียดฟิลด์ของตาราง ip_address	26
3-11	รายละเอียดฟิลด์ของตาราง semester	27
3-12	รายละเอียดฟิลด์ของตาราง pve_node	27
3-13	รายละเอียดฟิลด์ของตาราง instance_template	27
3-14	รายละเอียดฟิลด์ของตาราง instance_course	28
3-15	รายละเอียดฟิลด์ของตาราง instance_request	28
3-16	รายละเอียดฟิลด์ของตาราง instance	29
3-17	รายละเอียดฟิลด์ของตาราง instance_request_extends	29
4-1	สรุปผลการรวมการทดสอบ Midori	44
4-2	ผลการทดสอบรายเส้นทาง (Midori)	44
4-3	สรุปผลการรวมการทดสอบ Momoi API	45
4-4	ผลการทดสอบรายเส้นทาง (Momoi API)	45
4-5	สรุปผลการทดสอบการโคลน VM: Linked vs Full	46

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญ

การเรียนการสอนในภาควิชาเทคโนโลยีสารสนเทศมีความจำเป็นต้องใช้ระบบปฏิบัติการลินุกซ์ในหลายรายวิชา โดยเฉพาะในรายวิชาที่เกี่ยวข้องกับระบบเซิร์ฟเวอร์ การพัฒนาแอปพลิเคชัน และการจัดการโครงสร้างพื้นฐานด้านไอที อย่างไรก็ตาม การเตรียมสภาพแวดล้อมสำหรับการใช้งานระบบลินุกซ์นั้นมักมีความยุ่งยากและใช้เวลานาน ทั้งในด้านการติดตั้ง การตั้งค่า และการจัดการทรัพยากร ส่งผลให้การเรียนการสอนไม่สามารถดำเนินไปได้อย่างเต็มประสิทธิภาพ

ที่ผ่านมา นักศึกษาและอาจารย์ต้องใช้วิธีการจำลองระบบลินุกซ์ผ่านเครื่องเสมือน (Virtual Machine), ระบบย่อยของวินโดวส์สำหรับลินุกซ์ (WSL), หรือบริการคลาวด์จากภายนอก ซึ่งแต่ละวิธีมีข้อจำกัด เช่น ความยุ่งยากในการตั้งค่า ความไม่เสถียรของระบบ หรือค่าใช้จ่ายเพิ่มเติมในการใช้งานบริการจากผู้ให้บริการภายนอก โดยเฉพาะในการทำโครงการที่ต้องใช้เซิร์ฟเวอร์ นักศึกษามักต้องจัดหาเครื่องคอมพิวเตอร์ส่วนตัวหรือเซิร์ฟเวอร์ด้วยตนเอง ซึ่งเป็นภาระทั้งด้านเวลาและงบประมาณ

เพื่อแก้ไขปัญหาดังกล่าวข้างต้น โครงการนี้จึงมีแนวคิดในการพัฒนาแพลตฟอร์มคลัสเตอร์ภายในภาควิชา โดยใช้เทคโนโลยี Virtualization ร่วมกับระบบ cloud-init เพื่อสร้างเครื่องจำลองลินุกซ์ที่สามารถใช้งานได้อย่างรวดเร็วและง่ายดาย พร้อมทั้งมีระบบ Web Application สำหรับการจัดการและใช้งานเครื่องเสมือน ทั้งในส่วนของนักศึกษาและอาจารย์

1.2 วัตถุประสงค์ของโครงการ

- 1.2.1 เพื่อออกแบบและพัฒนาแพลตฟอร์มคลัสเตอร์เสมือน (Virtual Cluster Platform) ที่สามารถสร้างและจัดการเครื่องเสมือนลินุกซ์ได้อย่างรวดเร็วและมีประสิทธิภาพ สำหรับสนับสนุนการเรียนการสอนและการทำโครงการในภาควิชาเทคโนโลยีสารสนเทศ
- 1.2.2 เพื่อพัฒนา Web Application แบบ Role-Based Access Control (RBAC) ที่รองรับการใช้งานของทั้งนักศึกษาและอาจารย์ ในการร้องขอ จัดการ และเข้าถึงเครื่องเสมือน โดยคำนึงถึงความสะดวกในการใช้งาน เสถียรภาพของระบบ และความปลอดภัยในการเข้าถึงแพลตฟอร์ม

- 1.2.3 เพื่อเพิ่มประสิทธิภาพการใช้ทรัพยากรเซิร์ฟเวอร์ที่มีอยู่ภายในภาควิชา ผ่านการพัฒนา ระบบคลัสเตอร์แบบรวมศูนย์ ที่สามารถติดตาม วิเคราะห์ และบริหารจัดการทรัพยากร ได้อย่างเหมาะสม โดยมีระบบ Monitoring และ Cloud Storage ที่สามารถรองรับ การใช้งานในลักษณะ multi-user environment

1.3 ขอบเขตของการทำโครงการ

ภาคการศึกษาที่ 1/2568

- 1) จัดทำระบบการจัดการเครื่องคอมพิวเตอร์เซิร์ฟเวอร์
 - 1.1) จัดทำระบบ Hypervisor สำหรับการทำ Virtualization
 - 1.2) ระบบการจัดการเครื่องเสมือนที่อยู่ภายใต้ระบบ Hypervisor ด้วย API
 - 1.2.1) การสร้างเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.2) การแก้ไขข้อมูลเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.3) การลบเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.4) การจัดการสถานะเครื่องเสมือนบนระบบ Hypervisor
- 2) จัดทำระบบ Web Application สำหรับการเข้าใช้งานระบบ
 - 2.1) ระบบเข้าใช้งานสำหรับนักศึกษา
 - 2.1.1) ระบบการยืนยันตัวตนด้วย Google Account เพื่อเข้าใช้งานระบบด้วย โดเมน @email.kmutnb.ac.th และยืนยันภาควิชาที่นักศึกษาสังกัดด้วย รหัสนักศึกษาที่ปรากฏบนที่อยู่อีเมลล์
 - 2.1.2) ระบบยื่นคำร้องขอการใช้งานเครื่องเสมือนสำหรับการใช้งานในการเรียนปกติ หรือสำหรับการทำโครงการพิเศษ
 - 2.1.3) ระบบการจัดการเครื่องเสมือนและการเข้าใช้งานเครื่องเสมือนของนักศึกษา ที่ยังมีสถานะปกติ
 - 2.1.4) ระบบยื่นคำร้องขอต่ออายุการใช้งานเครื่องเสมือนที่มีอยู่เพื่อขอใช้งานเครื่องเสมือนต่อในภาคการศึกษาถัดไป
 - 2.2) ระบบเข้าใช้งานสำหรับอาจารย์หรือผู้ดูแลระบบ
 - 2.2.1) ระบบการยืนยันตัวตนด้วย Google Account เพื่อเข้าใช้งานระบบด้วยโดเมน @itm.kmutnb.ac.th และจำกัดการเข้าใช้ระบบเฉพาะบุคลากรภายในภาควิชา
 - 2.2.2) ระบบจัดการคำร้องขอใช้งานเครื่องเสมือนและคำร้องขอต่ออายุการใช้งานเครื่องเสมือนของนักศึกษา
 - 2.2.3) ระบบการสร้างเครื่องเสมือนสำหรับการใช้งานของอาจารย์หรือผู้ดูแลระบบ
 - 2.2.4) ระบบการจัดการและเข้าใช้งานเครื่องเสมือนของอาจารย์หรือผู้ดูแลระบบ

- 2.2.5) ระบบการจัดการเครื่องเสมือนของนักศึกษาในระบบ โดยสามารถเปลี่ยนแปลงสถานะของเครื่องเสมือนให้อยู่ในรูปแบบการจัดเก็บเครื่องเสมือนถาวรหรือการลบเครื่องเสมือนของนักศึกษา
- 2.2.6) ระบบการกำหนดช่วงเวลาของภาคการศึกษา เพื่อให้ระบบสามารถจัดหมวดหมู่ของเครื่องเสมือนในภาคการศึกษานั้น ๆ ได้
- 2.2.7) ระบบการจัดการรายชื่ออาจารย์หรือผู้ดูแลที่สังกัดภาควิชา

ภาคการศึกษาที่ 2/2568

- 1) จัดทำระบบ Reverse Proxy สำหรับการเข้าใช้งานพอร์ตต่าง ๆ ของเครื่องเสมือนจากระบบเครือข่ายสาธารณะได้
- 2) จัดทำระบบ Cloud Storage สำหรับการจัดเก็บไฟล์สำหรับการใช้งานของอาจารย์หรือผู้ดูแลระบบภายในภาควิชา
- 3) จัดทำระบบ Monitoring ที่จัดเก็บข้อมูล Trace, Metric และ Log โดยสามารถแสดงผลผ่าน Dashboard ได้
- 4) จัดทำระบบเพิ่มเติมบน Web Application
 - 4.1) ระบบการกำหนดค่า Port Forwarding บนระบบ Reverse Proxy เพื่อกำหนดพอร์ตที่ต้องการใช้งานจากระบบเครือข่ายภายนอก
 - 4.2) ระบบการเข้าใช้งาน Cloud Storage สำหรับอาจารย์หรือผู้ดูแลระบบ เพื่อใช้งานในการจัดเก็บไฟล์
- 5) ประเมินประสิทธิภาพของระบบ
 - 5.1) วัดค่า Provisioning Time วัดค่าเฉลี่ย (Mean) และ ค่ามัธยฐาน (Median) ของเวลาที่ใช้ในการสร้างเครื่องเสมือนใหม่ ตั้งแต่เริ่มต้นกระบวนการจนพร้อมใช้งานเปรียบเทียบกับวิธีการเดิม
 - 5.2) วัดค่า Concurrent User Performance ทดสอบความเร็วการตอบสนองของระบบเมื่อมีผู้ใช้งานพร้อมกันหลายราย วัดค่าการตอบสนองของระบบ (Response Time) เฉลี่ย (ms) และอัตราความล้มเหลว (Error Rate) (%)

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1.4.1 ได้ระบบแพลตฟอร์ม คลัสเตอร์ ที่สามารถใช้งานได้จริง ภายใน ภาควิชา เทคโนโลยีสารสนเทศ เพื่อสนับสนุนการเรียนการสอน และการทำโครงการ ของนักศึกษาและอาจารย์
- 1.4.2 ได้ระบบเครื่องเสมือน ลินุกซ์ ที่สามารถสร้าง และ จัดการได้อย่างรวดเร็วผ่าน Web Application โดยไม่ต้องติดตั้งระบบปฏิบัติการเพิ่มเติมบนเครื่องของผู้ใช้งาน

- 1.4.3 ได้ใช้ประโยชน์จากเครื่องเซิร์ฟเวอร์ที่มีอยู่ในภาควิชาอย่างเต็มประสิทธิภาพ โดยนำมาให้บริการในรูปแบบคลัสเตอร์สำหรับการเรียนการสอน
- 1.4.4 ได้โครงการที่สามารถนำไปใช้งานจริงภายในภาควิชา และสามารถต่อยอดพัฒนาเพิ่มเติมในอนาคต
- 1.4.5 ได้โครงการอื่น ๆ ที่สามารถใช้งานได้จริงในภาควิชา โดยนำระบบที่นักศึกษาท่านอื่นที่พัฒนาในโครงการมาติดตั้งบนแพลตฟอร์มคลัสเตอร์

บทที่ 2

แนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดพื้นฐานเกี่ยวกับโครงงาน

2.1.1 ความสำคัญของการเรียนการสอนด้วยระบบปฏิบัติการลินุกซ์ในการศึกษาด้านเทคโนโลยีสารสนเทศ

การเรียนการสอนในสาขาเทคโนโลยีสารสนเทศ โดยเฉพาะในระดับอุดมศึกษา มีความจำเป็นอย่างยิ่งที่จะต้องให้นักศึกษาได้ฝึกฝนการใช้งานระบบปฏิบัติการลินุกซ์ เนื่องจากระบบปฏิบัติการดังกล่าวเป็นพื้นฐานสำคัญในการทำงานด้านเซิร์ฟเวอร์ การพัฒนาระบบ การจัดการโครงสร้างพื้นฐานด้านไอที และเทคโนโลยีคลาวด์ที่กำลังเป็นที่ต้องการของตลาดแรงงานในปัจจุบัน (Anderson, 2008)

การเรียนรู้และฝึกฝนการใช้งานระบบลินุกซ์มีความสำคัญในหลายประการ ได้แก่

- **พัฒนาทักษะการจัดการระบบ** ช่วยให้นักศึกษาเข้าใจและสามารถจัดการระบบปฏิบัติการที่ใช้กันอย่างแพร่หลายในโลกธุรกิจ โดยเฉพาะในด้านเซิร์ฟเวอร์และระบบคลาวด์
- **เสริมสร้างความเข้าใจในการทำงานของระบบคอมพิวเตอร์** การใช้งานลินุกซ์ผ่าน Command Line Interface ช่วยให้นักศึกษาเข้าใจการทำงานของระบบในระดับที่ลึกซึ้ง
- **เตรียมความพร้อมสู่การทำงาน** ระบบลินุกซ์เป็นมาตรฐานในอุตสาหกรรมเทคโนโลยี การฝึกฝนการใช้งานจะช่วยให้นักศึกษามีความพร้อมในการทำงานจริง
- **สนับสนุนการเรียนรู้เทคโนโลยีใหม่** หลายเทคโนโลยีสมัยใหม่ เช่น Docker, Kubernetes, และระบบ DevOps ต่างๆ ต้องอาศัยความรู้พื้นฐานเกี่ยวกับลินุกซ์ (Lee & Chen, 2021)

อย่างไรก็ตาม การจัดเตรียมสภาพแวดล้อมสำหรับการเรียนการสอนด้วยระบบลินุกซ์นั้นมักเผชิญกับความท้าทายหลายประการ ได้แก่

- **ความยุ่งยากในการติดตั้งและตั้งค่า** การติดตั้งระบบลินุกซ์บนเครื่องคอมพิวเตอร์ส่วนบุคคลของนักศึกษามักต้องใช้เวลานานและมีความซับซ้อน โดยเฉพาะสำหรับผู้ที่ไม่มีความชำนาญ
- **ข้อจำกัดด้านฮาร์ดแวร์** เครื่องคอมพิวเตอร์ส่วนบุคคลของนักศึกษาอาจมีข้อจำกัดด้านหน่วยความจำหรือพื้นที่จัดเก็บข้อมูล ทำให้การใช้งานเครื่องเสมือนไม่เสถียร
- **ความไม่สม่ำเสมอของสภาพแวดล้อม** นักศึกษาแต่ละคนอาจมีการตั้งค่าและเวอร์ชันของซอฟต์แวร์ที่แตกต่างกัน ทำให้เกิดปัญหาในการเรียนการสอนแบบเดียวกัน
- **การบำรุงรักษาและแก้ไขปัญหา** เมื่อเกิดปัญหาทางเทคนิค อาจารย์และนักศึกษาต้องใช้

เวลามากในการแก้ไขปัญหาแทนที่จะมุ่งเน้นไปที่การเรียนรู้เนื้อหาหลัก

- **ค่าใช้จ่ายในการใช้บริการภายนอก** การใช้บริการคลาวด์จากผู้ให้บริการภายนอกมักมีค่าใช้จ่ายที่สูง โดยเฉพาะเมื่อต้องใช้งานตลอดภาคการศึกษา

2.1.2 บทบาทของเทคโนโลยี Virtualization และ Cloud Computing ในการศึกษา

ในยุคที่เทคโนโลยีสารสนเทศพัฒนาอย่างรวดเร็ว การนำเทคโนโลยี Virtualization และ Cloud Computing มาประยุกต์ใช้ในการศึกษาได้กลายเป็นแนวทางที่สำคัญในการแก้ไขปัญหาและเพิ่มประสิทธิภาพการเรียนการสอน โดยเฉพาะอย่างยิ่งในสาขาเทคโนโลยีสารสนเทศที่ต้องการสภาพแวดล้อมที่หลากหลายและซับซ้อนสำหรับการฝึกปฏิบัติ (Clark & Kwinn, 2016)

เทคโนโลยี Virtualization และ Cloud Computing มีบทบาทสำคัญในการเพิ่มประสิทธิภาพการเรียนการสอนหลายประการ ได้แก่

- **ลดเวลาและความซับซ้อนในการตั้งค่าระบบ** เทคโนโลยีเครื่องเสมือนสามารถสร้างสภาพแวดล้อมที่พร้อมใช้งานได้อย่างรวดเร็ว โดยไม่ต้องให้นักศึกษาแต่ละคนติดตั้งและตั้งค่าระบบด้วยตนเอง
- **เพิ่มความยืดหยุ่นในการจัดการทรัพยากร** สามารถปรับเปลี่ยนขนาดและสเปกของเครื่องเสมือนตามความต้องการของแต่ละรายวิชาหรือโครงการ โดยไม่ต้องลงทุนซื้อฮาร์ดแวร์เพิ่มเติม
- **รองรับการเรียนรู้แบบ Remote Learning** นักศึกษาสามารถเข้าถึงและใช้งานระบบได้จากที่ใดก็ได้ トラบใดที่มีการเชื่อมต่ออินเทอร์เน็ต ทำให้การเรียนการสอนไม่ถูกจำกัดด้วยสถานที่และเวลา
- **สร้างความสม่ำเสมอของสภาพแวดล้อม** ทุกคนจะได้ใช้งานระบบที่มีการตั้งค่าเหมือนกัน ลดปัญหาที่เกิดจากความแตกต่างของสภาพแวดล้อมในเครื่องของแต่ละคน
- **ประหยัดค่าใช้จ่าย** การใช้ทรัพยากรแบบรวมศูนย์ช่วยลดค่าใช้จ่ายในการซื้อและบำรุงรักษาฮาร์ดแวร์ รวมถึงลดค่าใช้จ่ายสำหรับนักศึกษาในการจัดหาเครื่องมือสำหรับการเรียน

แนวคิดของแพลตฟอร์มคลัสเตอร์สำหรับการศึกษาจึงเป็นการต่อยอดจากข้อดีของเทคโนโลยีเหล่านี้ โดยการสร้างระบบที่สามารถ

- **จัดการเครื่องเสมือนแบบรวมศูนย์** ผ่านระบบ Web Application ที่ใช้งานง่าย ทำให้อาจารย์และนักศึกษาสามารถร้องขอ สร้าง และจัดการเครื่องเสมือนได้โดยไม่ต้องมีความรู้ลึกด้านระบบ
- **ใช้ประโยชน์จากทรัพยากรที่มีอยู่อย่างเต็มที่** การรวมเครื่องเซิร์ฟเวอร์หลายเครื่องเข้าเป็นคลัสเตอร์เดียว ช่วยให้สามารถใช้งานทรัพยากรได้อย่างมีประสิทธิภาพสูงสุด
- **รองรับการใช้งานหลากหลายรูปแบบ** ทั้งการเรียนการสอนปกติ การทำโครงการ การวิจัย

และการพัฒนาระบบต่างๆ ภายในภาควิชา

- **มีระบบติดตามและบริหารจัดการ** ด้วยระบบ Monitoring ที่ช่วยในการติดตามการใช้งาน และแก้ไขปัญหาได้อย่างรวดเร็ว

2.2 แนวคิดและทฤษฎีที่เกี่ยวข้องกับแพลตฟอร์มคลัสเตอร์เสมือน

2.2.1 เทคโนโลยี Virtualization

Virtualization Technology เป็นเทคนิคการจำลองซอฟต์แวร์หรือฮาร์ดแวร์บนซอฟต์แวร์อื่น โดยสภาพแวดล้อมจำลองนี้เรียกว่าเครื่องเสมือน (Virtual Machine) ซึ่งเป็นเทคโนโลยีพื้นฐานสำคัญที่ทำให้สามารถแบ่งทรัพยากรฮาร์ดแวร์เครื่องเดียวออกเป็นหลายๆ ระบบที่ทำงานได้อิสระจากกัน (Garcia & Rodriguez, 2023)

Hypervisor และประเภทของ Virtualization

- **Type 1 Hypervisor (Bare-metal)** เป็น Hypervisor ที่ทำงานโดยตรงบนฮาร์ดแวร์ โดยไม่ต้องผ่านระบบปฏิบัติการหลัก เช่น VMware vSphere, Microsoft Hyper-V, และ Proxmox VE ซึ่งมีประสิทธิภาพสูงและเหมาะสำหรับการใช้งานในระดับองค์กร
- **Type 2 Hypervisor (Hosted)** เป็น Hypervisor ที่ทำงานบนระบบปฏิบัติการหลัก เช่น VMware Workstation, VirtualBox ซึ่งง่ายต่อการใช้งานแต่มีประสิทธิภาพต่ำกว่า Type 1
- **Container-based Virtualization** เป็นรูปแบบของ Virtualization ที่แบ่งปันระบบปฏิบัติการเดียวกัน เช่น Docker, LXC ซึ่งมีความเร็วและประสิทธิภาพสูงกว่าแต่มีความแยกตัวน้อยกว่าเครื่องเสมือนแบบเต็มรูปแบบ

ข้อดีของเทคโนโลยี Virtualization

- **การใช้ทรัพยากรอย่างมีประสิทธิภาพ** สามารถใช้ทรัพยากรฮาร์ดแวร์ได้เต็มประสิทธิภาพ เนื่องจากสามารถรันหลายระบบพร้อมกันได้บนเครื่องเดียว
- **ความยืดหยุ่นในการจัดการ** สามารถสร้าง ลบ หรือปรับแต่งเครื่องเสมือนได้อย่างรวดเร็ว โดยไม่ต้องแตะต้องฮาร์ดแวร์จริง
- **การแยกตัวและความปลอดภัย** แต่ละเครื่องเสมือนทำงานแยกจากกัน หากเครื่องหนึ่งเกิดปัญหาจะไม่ส่งผลกระทบต่อเครื่องอื่น
- **ความสะดวกในการบำรุงรักษา** สามารถสำรองข้อมูล ย้าย หรือกู้คืนระบบได้ง่าย โดยไม่ต้องหยุดการทำงานของเครื่องอื่น

2.2.2 Cloud Computing และ Container Orchestration

Cloud Computing เป็นแนวทางการประมวลผลแบบคลาวด์ที่ช่วยให้สามารถเข้าถึงทรัพยากรการประมวลผลที่กำหนดค่าได้ตามต้องการ ซึ่งสามารถจัดเตรียมและปล่อยทรัพยากรได้อย่างรวดเร็วโดยใช้ความพยายามในการจัดการหรือการโต้ตอบระหว่างผู้ให้บริการน้อยที่สุด โดยมีรูปแบบการให้บริการหลัก ได้แก่

รูปแบบการให้บริการ Cloud Computing

- **Infrastructure as a Service (IaaS)** การให้บริการโครงสร้างพื้นฐานด้านไอที เช่น เครื่องเสมือน พื้นที่จัดเก็บข้อมูล เครือข่าย โดยผู้ใช้สามารถควบคุมระบบปฏิบัติการและแอปพลิเคชันได้
- **Platform as a Service (PaaS)** การให้บริการแพลตฟอร์มสำหรับการพัฒนาและปรับใช้แอปพลิเคชัน โดยไม่ต้องจัดการโครงสร้างพื้นฐานด้านล่าง
- **Software as a Service (SaaS)** การให้บริการซอฟต์แวร์ที่พร้อมใช้งานผ่านอินเทอร์เน็ต โดยไม่ต้องติดตั้งหรือจัดการด้วยตนเอง

Container และ Container Orchestration

Container คือการรวมแอปพลิเคชันและรหัสไบনারี ไลบรารี และการกำหนดค่าเข้าด้วยกันในขณะที่ยังแบ่งปันอิมเมจระบบปฏิบัติการโฮสต์ ดังนั้นคอนเทนเนอร์จึงแบ่งปันทรัพยากรและดำเนินการไมโครเซอร์วิสขนาดเล็ก โปรแกรมซอฟต์แวร์ และแอปพลิเคชันที่ครอบคลุมยิ่งขึ้นได้อย่างมีประสิทธิภาพ ด้วยค่าใช้จ่ายด้านการจัดการน้อยกว่าเครื่องเสมือน

- **ข้อดีของ Container** มีขนาดเล็ก เริ่มต้นเร็ว ใช้ทรัพยากรน้อย และสามารถพกพาได้ง่ายเนื่องจากบรรจุทุกสิ่งที่จำเป็นสำหรับการทำงานของแอปพลิเคชัน
- **Docker** เป็นแพลตฟอร์ม Container ที่ได้รับความนิยมสูงสุด ช่วยให้การสร้าง จัดการ และปรับใช้ Container เป็นเรื่องง่าย

Container Orchestration คือการปรับใช้และจัดการแอปพลิเคชันที่เป็น Container ที่อยู่ในคลัสเตอร์ที่เชื่อมต่อถึงกัน โดยมีโหนดที่คอยควบคุมการทำงานที่เรียกว่าโหนดมาสเตอร์ (Master node) ซึ่งมีหน้าที่ดูแลคลัสเตอร์และจัดการการปรับใช้คอนเทนเนอร์บนโหนดเวิร์กเกอร์ (Worker node)

- **การจัดการทรัพยากรอัตโนมัติ** ระบบจะจัดสรรทรัพยากรให้กับ Container ตามความต้องการและสถานะของคลัสเตอร์
- **High Availability** สามารถกระจาย Container ไปยังหลายโหนดเพื่อป้องกันการหยุดทำงานเมื่อโหนดใดโหนดหนึ่งเกิดปัญหา
- **Auto Scaling** สามารถปรับขนาดของแอปพลิเคชันขึ้นลงอัตโนมัติตามปริมาณการใช้งาน
- **Service Discovery และ Load Balancing** ช่วยในการเชื่อมต่อ Container ต่างๆ และ

กระจายโหลดการทำงาน

2.2.3 Web Application และเทคโนโลยีสนับสนุน

Web Application คือการพัฒนาระบบงานบนเว็บ ข้อมูลต่างๆ ในระบบสามารถเข้าถึงได้ทั้งเครือข่ายภายในหรือเครือข่ายสาธารณะ ซึ่งทำให้ใช้งานง่ายผ่านเว็บเบราว์เซอร์ โดยในการใช้งานนั้นไม่จำเป็นต้องติดตั้งโปรแกรมใดๆ เพิ่มเติม ทำให้เหมาะสำหรับการใช้งานในสภาพแวดล้อมที่มีผู้ใช้งานหลากหลาย

คุณสมบัติสำคัญของ Web Application สำหรับการจัดการระบบคลัสเตอร์

- **Cross-platform Compatibility** สามารถใช้งานได้บนระบบปฏิบัติการต่างๆ โดยไม่ต้องพัฒนาแยกสำหรับแต่ละแพลตฟอร์ม
- **Centralized Management** ช่วยให้อาจจัดการระบบจากจุดเดียว ลดความซับซ้อนในการดูแลระบบหลายเครื่อง
- **Role-Based Access Control (RBAC)** สามารถกำหนดสิทธิ์การเข้าถึงตามบทบาทของผู้ใช้ เช่น นักศึกษา อาจารย์ และผู้ดูแลระบบ
- **Real-time Monitoring** สามารถแสดงสถานะของระบบแบบเรียลไทม์ ช่วยในการติดตามและแก้ไขปัญหาได้รวดเร็ว

Cloud-Init และการตั้งค่าเครื่องเสมือนอัตโนมัติ

Cloud-Init คือการตั้งค่าที่ถูกรับรองไว้ล่วงหน้าสำหรับการจัดเตรียมเครื่องจำลองใหม่โดยอัตโนมัติ ซึ่งรวมถึงการสร้างข้อมูลประจำตัวผู้ใช้และการตั้งค่าระบบขั้นพื้นฐาน ด้วยการสร้างไฟล์ cloud-config ที่ปรับให้เหมาะกับเครื่องเสมือนแต่ละเครื่องอย่างไดนามิก

- **การตั้งค่าผู้ใช้งานอัตโนมัติ** สามารถสร้างบัญชีผู้ใช้ กำหนดรหัสผ่าน และตั้งค่า SSH keys ได้โดยอัตโนมัติ
- **การติดตั้งซอฟต์แวร์เริ่มต้น** สามารถกำหนดให้ติดตั้งแพ็คเกจต่างๆ ที่จำเป็นสำหรับการใช้งานเบื้องต้น
- **การตั้งค่าเครือข่าย** สามารถกำหนดค่า IP Address, DNS, และการตั้งค่าเครือข่ายอื่นๆ ได้
- **การรันสคริปต์เริ่มต้น** สามารถกำหนดให้รันคำสั่งหรือสคริปต์ต่างๆ หลังจากบูตระบบขึ้นมาแล้ว

Reverse Proxy และการจัดการการเข้าถึง

Reverse Proxy คือตัวกลางที่ส่งข้อมูลเครือข่ายที่แลกเปลี่ยนข้อมูลระหว่างไคลเอนต์และเซิร์ฟเวอร์ โดยจะรับคำขอจากไคลเอนต์บนเครือข่ายสาธารณะ (อินเทอร์เน็ต) ส่งต่อคำขอไปยังเซิร์ฟเวอร์ในเครือข่ายภายใน จากนั้นจึงส่งคำตอบที่สร้างโดยเซิร์ฟเวอร์ไปยังไคลเอนต์

- **Port Forwarding** ช่วยให้สามารถเข้าถึงบริการต่างๆ ในเครื่องเสมือนจากภายนอกได้โดยผ่านพอร์ตที่กำหนด
- **Load Balancing** สามารถกระจายโหลดไปยังเซิร์ฟเวอร์หลายตัวเพื่อเพิ่มประสิทธิภาพ
- **SSL Termination** สามารถจัดการการเข้ารหัส SSL/TLS ได้ที่ตัว Proxy โดยไม่ต้องตั้งค่าในแต่ละเซิร์ฟเวอร์
- **Security** ช่วยป้องกันการเข้าถึงโดยตรงไปยังเซิร์ฟเวอร์ภายใน และสามารถกำหนดกฎการเข้าถึงได้

สรุปได้ว่า การนำเทคโนโลยี Virtualization, Cloud Computing, Container Orchestration, Web Application และเทคโนโลยีสนับสนุนต่างๆ มาใช้ร่วมกันในการพัฒนาแพลตฟอร์มคลัสเตอร์สำหรับการศึกษา จะช่วยให้สามารถสร้างสภาพแวดล้อมการเรียนการสอนที่มีประสิทธิภาพ ยืดหยุ่น และตอบสนองความต้องการของผู้ใช้งานได้หลากหลายรูปแบบ โดยเฉพาะอย่างยิ่งในสาขาเทคโนโลยีสารสนเทศที่ต้องการการฝึกปฏิบัติในสภาพแวดล้อมที่เสมือนจริง

2.3 รายงานการค้นคว้า การศึกษา หรืองานวิจัยที่เกี่ยวข้อง

2.3.1 การพัฒนาระบบ Virtualization สำหรับการศึกษา

การศึกษาวิจัยเรื่อง ”Guide to Security for Full Virtualization Technologies” โดย Karen Scarfone, Murugiah Souppaya, และ Paul Hoffman ได้นำเสนอแนวทางและมาตรฐานด้านความปลอดภัยสำหรับเทคโนโลยี Virtualization แบบเต็มรูปแบบ (Karen Scarfone, 2011). งานวิจัยนี้มีความสำคัญต่อการพัฒนาแพลตฟอร์มคลัสเตอร์เนื่องจากให้ข้อมูลเกี่ยวกับการจำแนกประเภทของ Virtualization ตามชั้นสถาปัตยกรรมคอมพิวเตอร์ และแนวทางการรักษาความปลอดภัยที่จำเป็นซึ่งเป็นองค์ความรู้พื้นฐานสำคัญในการออกแบบระบบที่ปลอดภัยและเหมาะสมสำหรับสภาพแวดล้อมการศึกษา

2.3.2 การประยุกต์ใช้ Cloud Computing ในการศึกษา

Danairat T. ได้นำเสนอเอกสารเรื่อง ”Cloud Computing Technology The Architecture Overview” ซึ่งให้ภาพรวมของสถาปัตยกรรม Cloud Computing และแนวคิดการประมวลผลแบบคลาวด์ที่ช่วยให้สามารถเข้าถึงทรัพยากรการประมวลผลที่กำหนดค่าได้ตามต้องการ (T., 2015). งานศึกษานี้เป็นแนวทางสำคัญในการทำความเข้าใจรูปแบบการให้บริการ เช่น IaaS, PaaS, และ SaaS ซึ่งเป็นแนวคิดที่นำมาประยุกต์ใช้ในการพัฒนาแพลตฟอร์มคลัสเตอร์เพื่อให้บริการเครื่องเสมือนแก่ผู้ใช้งานในสถาบันการศึกษา

2.3.3 การพัฒนาระบบติดตามความก้าวหน้าสำหรับเอกสารโครงการพิเศษ

Rawiporn Suamsiri และ Kamonwan Janmanee ได้พัฒนาระบบ **"Progress Tracking System for Special Project Documents"** ซึ่งเป็นระบบ Web Application ที่ช่วยในการติดตามความก้าวหน้าของการจัดทำเอกสารโครงการพิเศษ (Suamsiri & Janmanee, 2024). งานวิจัยนี้แสดงให้เห็นถึงแนวทางการพัฒนา Web Application ที่ใช้งานง่ายผ่านเว็บเบราว์เซอร์โดยไม่จำเป็นต้องติดตั้งโปรแกรมเพิ่มเติม ซึ่งเป็นแนวคิดที่สอดคล้องกับการพัฒนาส่วนติดต่อผู้ใช้งานสำหรับแพลตฟอร์มคลัสเตอร์ในโครงการนี้

2.3.4 การใช้งาน Container Technologies ในสถาปัตยกรรมต่างๆ

งานวิจัยของ SHAHIDULLAH KAISER, MD.SADUNHAQ, ALI AMAN TOSUN, และ TURGAY KORKMAZ เรื่อง **"Container Technologies for ARM Architecture: A Comprehensive Survey of the State-of-the-Art"** นำเสนอการสำรวจเทคโนโลยี Container บนสถาปัตยกรรม ARM อย่างครอบคลุม (Kaiser et al., 2022). แสดงให้เห็นถึงการพัฒนาและการใช้งาน Container ที่มีประสิทธิภาพสูง ใช้ทรัพยากรน้อย และสามารถดำเนินการไมโครเซอร์วิสได้อย่างมีประสิทธิภาพ ซึ่งเป็นข้อมูลสำคัญในการออกแบบระบบที่ใช้ Container เป็นส่วนหนึ่งของโครงสร้างพื้นฐาน

2.3.5 Container Orchestration ในอุตสาหกรรมยานยนต์

Naresh Nayak, Dennis Grewe, และ Sebastian Schildt ได้นำเสนอการศึกษาเรื่อง **"Automotive Container Orchestration: Requirements, Challenges and Open Directions"** ซึ่งอธิบายแนวคิดของ Container Orchestration ในการปรับใช้และจัดการแอปพลิเคชันที่เป็น Container ในคลัสเตอร์ (Nayak et al., 2023). งานวิจัยนี้ให้ความรู้เกี่ยวกับการทำงานของโหนดมาสเตอร์และโหนดเวิร์กเกอร์ รวมถึงคุณสมบัติสำคัญ เช่น High Availability, Auto Scaling, และ Load Balancing ซึ่งเป็นแนวคิดที่นำมาประยุกต์ใช้ในการพัฒนาระบบจัดการคลัสเตอร์ในโครงการนี้

2.3.6 การสร้างโครงสร้างพื้นฐานคลาวด์สำหรับการจัดตารางเครื่องเสมือน

Jonathan Chya และ Xunfei Jiang ได้นำเสนอการศึกษาเรื่อง **"Building a Cloud Infrastructure for Virtual Machine Scheduling in Datacenters"** ซึ่งเป็นการศึกษาเกี่ยวกับการสร้างโครงสร้างพื้นฐานคลาวด์สำหรับการจัดตารางเครื่องเสมือนในศูนย์ข้อมูล รวมถึงการใช้ Cloud-Init ในการจัดเตรียมเครื่องเสมือนใหม่โดยอัตโนมัติ (Chya & Jiang, 2024). งานวิจัยนี้แสดงให้เห็นถึงการใช้ไฟล์ cloud-config ที่ปรับให้เหมาะกับเครื่องเสมือนแต่ละเครื่องอย่างไดนามิก ซึ่งช่วยเพิ่มประสิทธิภาพด้านพลังงาน ความเร็ว และการปรับแต่งกระบวนการจัดเตรียมเครื่องเสมือนได้อย่างมาก

2.3.7 การพัฒนาสถาปัตยกรรม Reverse Proxy

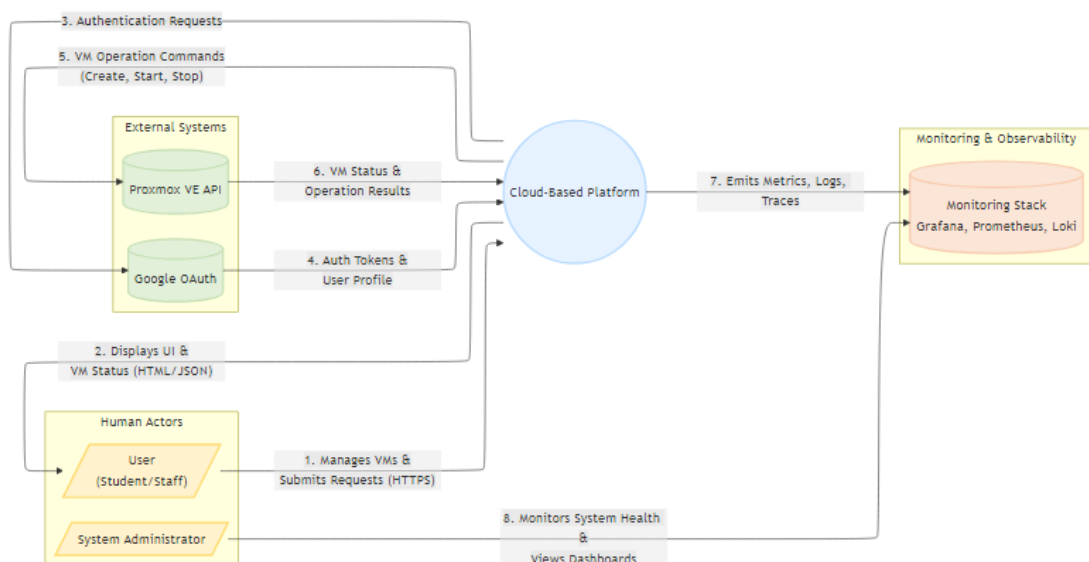
Yu TAO และ Gang CHEN ได้เผยแพร่งานวิจัยเรื่อง **”An Extensible Universal Reverse Proxy Architecture”** ซึ่งนำเสนอสถาปัตยกรรม Reverse Proxy ที่มีความยืดหยุ่นและสามารถขยายได้ (Tao & Chen, 2016). งานวิจัยนี้อธิบายการทำงานของ Reverse Proxy ในการรับคำขอจากไคลเอนต์บนเครือข่ายสาธารณะและส่งต่อไปยังเซิร์ฟเวอร์ในเครือข่ายภายใน รวมถึงความสามารถในการแคชทรัพยากรเพื่อเพิ่มประสิทธิภาพ ซึ่งเป็นแนวคิดสำคัญที่นำมาใช้ในการพัฒนาระบบ Port Forwarding และการจัดการการเข้าถึงเครื่องเสมือนจากภายนอกในโครงการนี้

บทที่ 3

วิธีการดำเนินงานและการออกแบบระบบ

3.1 การออกแบบสถาปัตยกรรมระบบ

3.1.1 สถาปัตยกรรมระบบโดยรวม

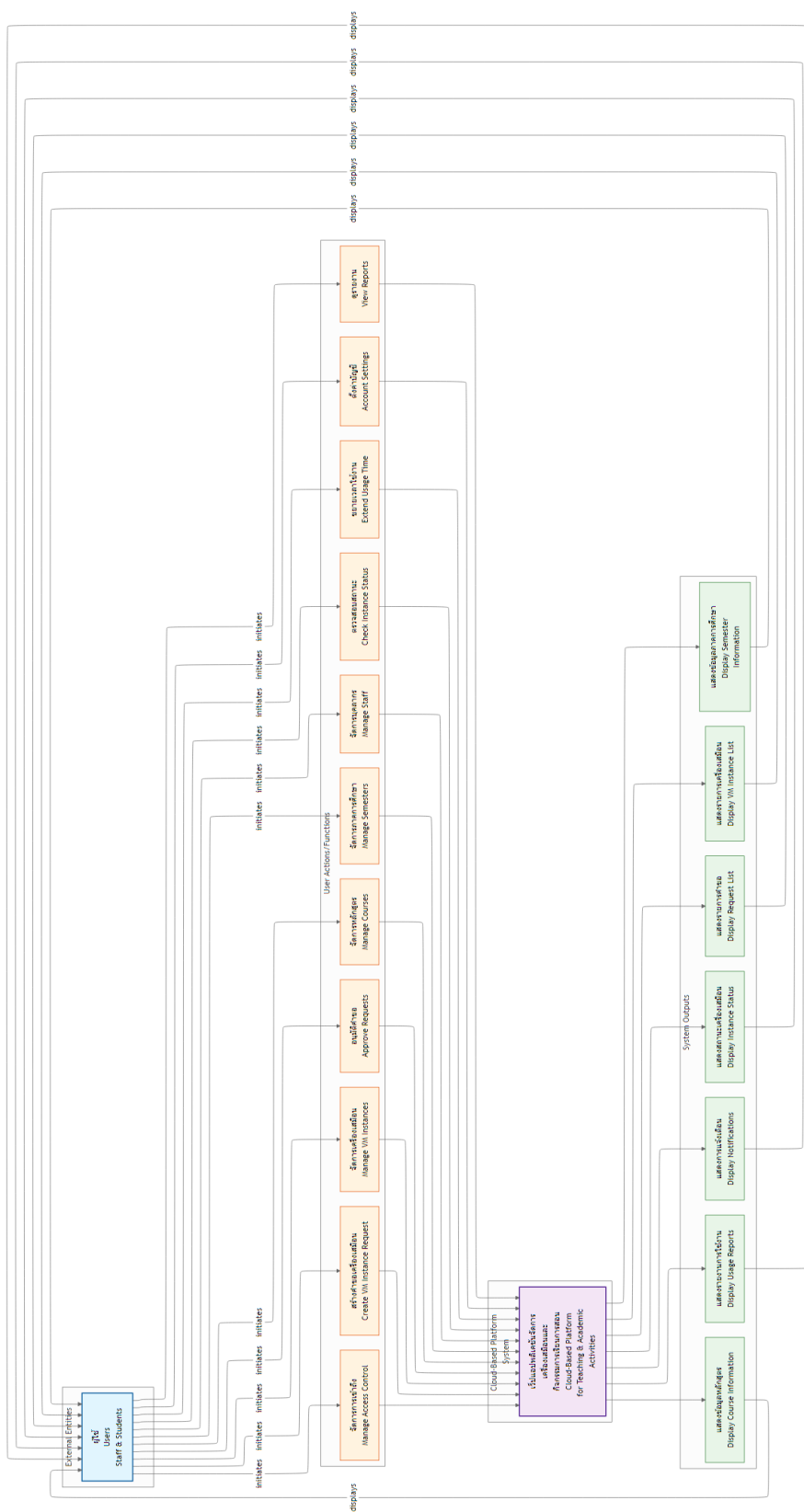


ภาพที่ 3-1 แผนภาพการทำงานและลำดับขั้นตอนโดยรวมของโปรแกรม

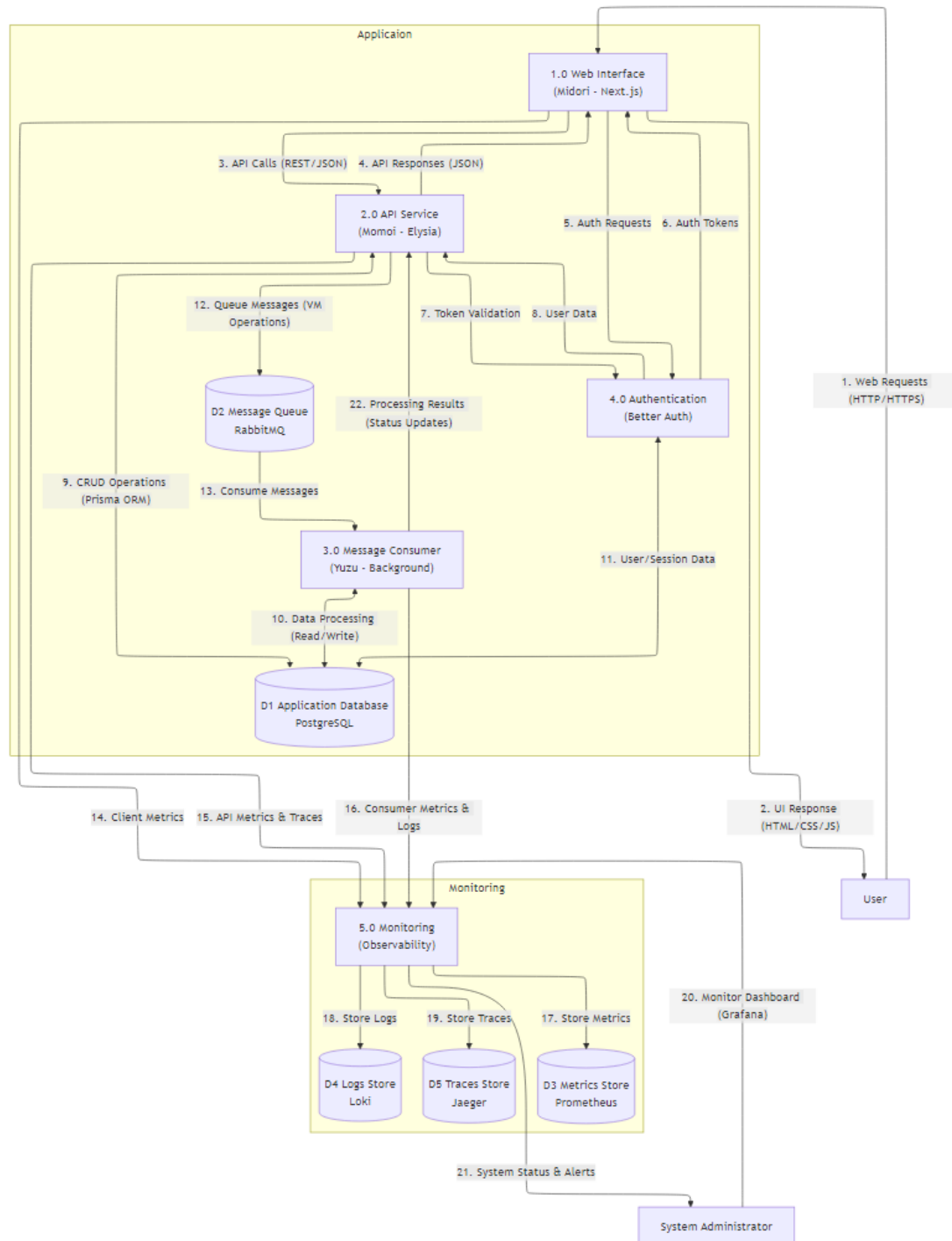
ระบบแพลตฟอร์มคลัสเตอร์เสมือนได้รับการออกแบบในรูปแบบ Microservices Architecture โดยแบ่งเป็นส่วนประกอบหลัก 3 ส่วน คือ

- **Frontend** เว็บแอปพลิเคชันสำหรับผู้ใช้งาน พัฒนาด้วย Next.js และ React
- **Backend** เซิร์ฟเวอร์ API สำหรับการจัดการข้อมูลและการยืนยันตัวตน พัฒนาด้วย Elysia.js
- **Instance Management (VM Management)** เซอร์วิสจัดการเครื่องเสมือนและการสื่อสารกับ Proxmox VE

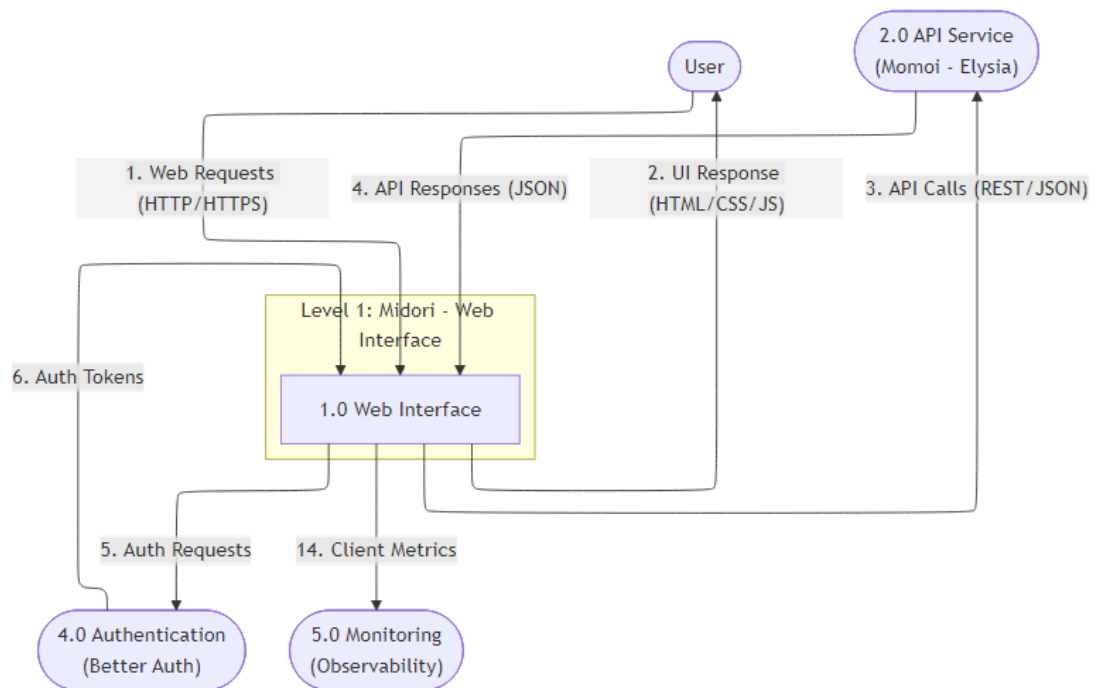
จากภาพที่ 3-2 แสดงการเชื่อมต่อและการสื่อสารระหว่างส่วนประกอบต่าง ๆ ของระบบ โดยผู้ใช้งานเข้าถึงระบบผ่านเว็บแอปพลิเคชันซึ่งจะติดต่อกับ Backend API และระบบจัดการเครื่องเสมือน เพื่อให้บริการที่สมบูรณ์ตามขั้นตอนรวมในภาพที่ 3-1 และรายละเอียดการไหลของข้อมูลตามภาพที่ 3-3 – 3-6



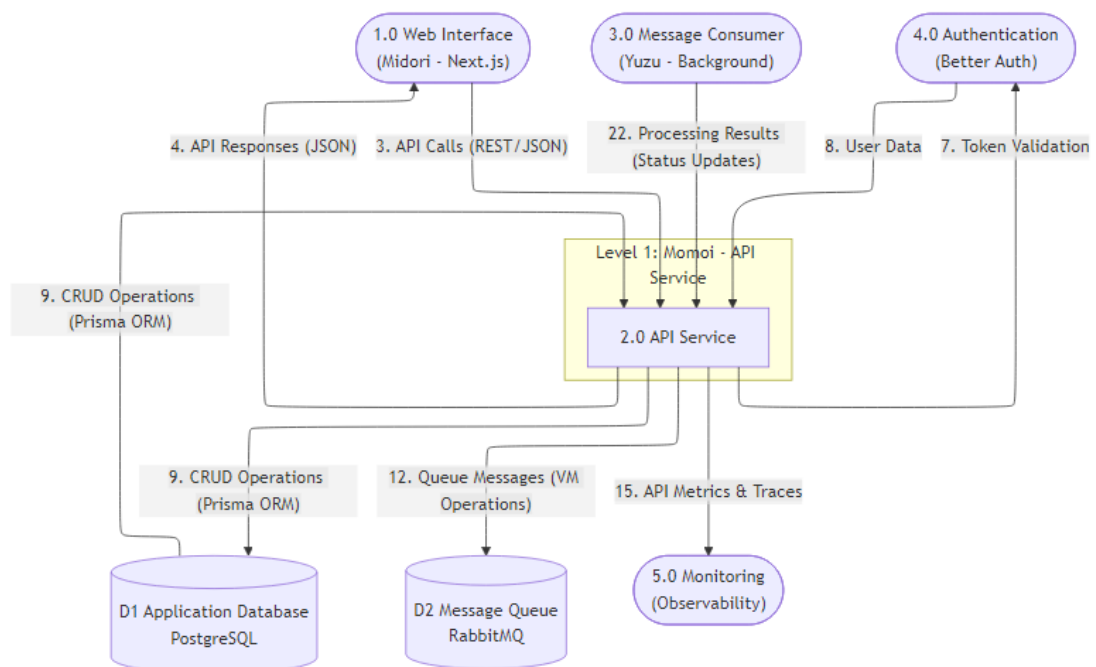
ภาพที่ 3-2 แผนภาพบริบทของระบบแพลตฟอร์มคลาวด์สำหรับ



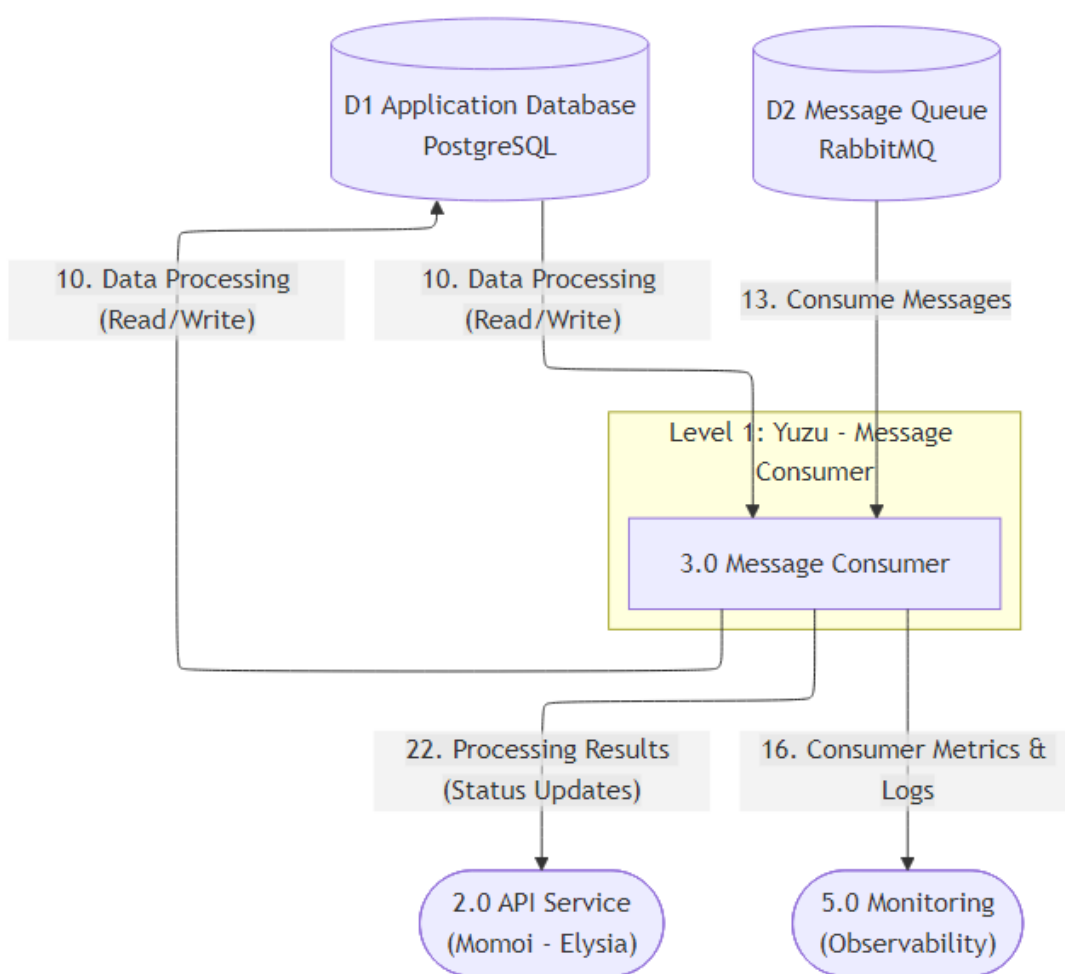
ภาพที่ 3-3 แผนภาพการไหลของข้อมูลระดับ 0



ภาพที่ 3-4 แผนภาพการไหลของข้อมูลระดับที่ 1 (Frontend)



ภาพที่ 3-5 แผนภาพการไหลของข้อมูลระดับที่ 1 (Backend)



ภาพที่ 3-6 แผนภาพการไหลของข้อมูลระดับที่ 1 (VM Management)

3.1.2 การออกแบบหน้าจอการทำงาน

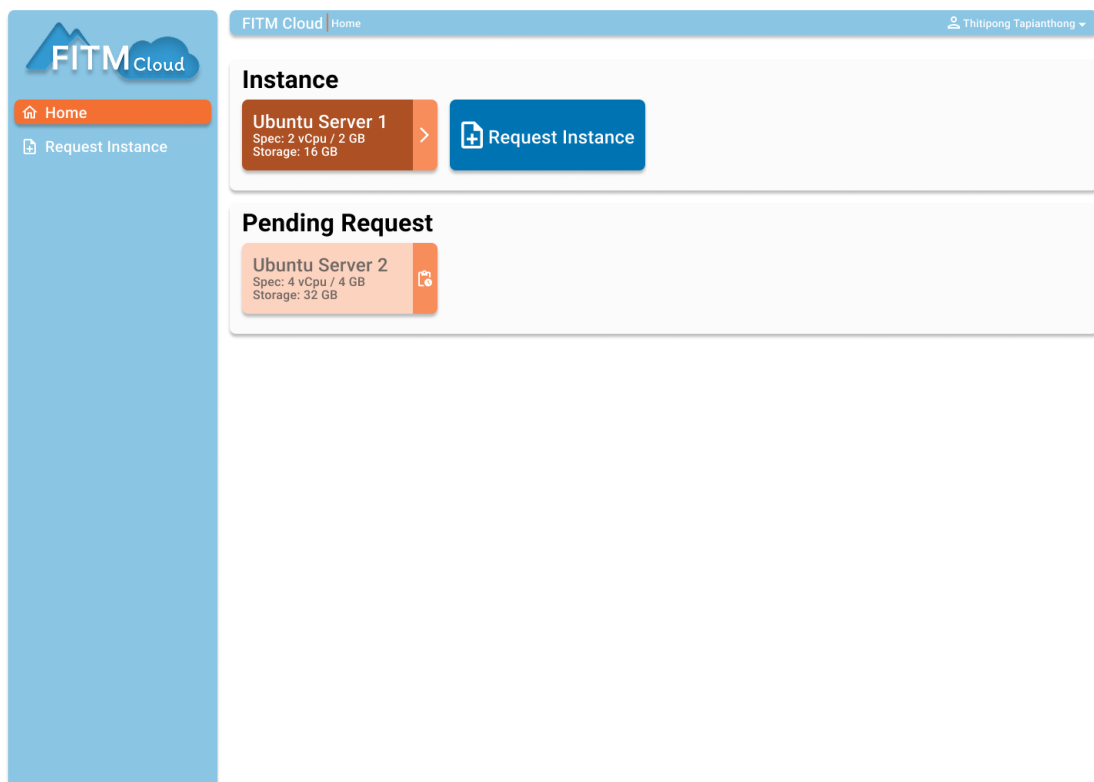
การออกแบบหน้าจอการทำงานของระบบแพลตฟอร์มคลัสเตอร์เสมือนได้รับการออกแบบให้มีความสะดวกในการใช้งานและการเข้าถึงข้อมูลที่จำเป็น โดยแบ่งออกเป็นหน้าจอหลัก ๆ ดังนี้

- **หน้าจอเข้าสู่ระบบ (Login Page)** หน้าจอสำหรับผู้ใช้ในการเข้าสู่ระบบผ่านการยืนยันตัวตน
- **หน้าจอแดชบอร์ด (Dashboard)** แสดงภาพรวมสถานะเครื่องเสมือน รายการคำร้องขอ และข้อมูลผู้ใช้
- **หน้าจอจัดการเครื่องเสมือน (Instance Management)** สำหรับผู้ใช้ในการดูรายละเอียดเครื่องเสมือน เช่น สถานะ การเชื่อมต่อ และการจัดการเครื่อง
- **หน้าจอคำร้องขอเครื่องเสมือน (Instance Request)** สำหรับผู้ใช้ในการส่งคำร้องขอการใช้งานเครื่องเสมือน
- **หน้าจอจัดการคำร้องขอ (Request Management)** สำหรับเจ้าหน้าที่ในการตรวจสอบและอนุมัติคำร้องขอเครื่องเสมือน

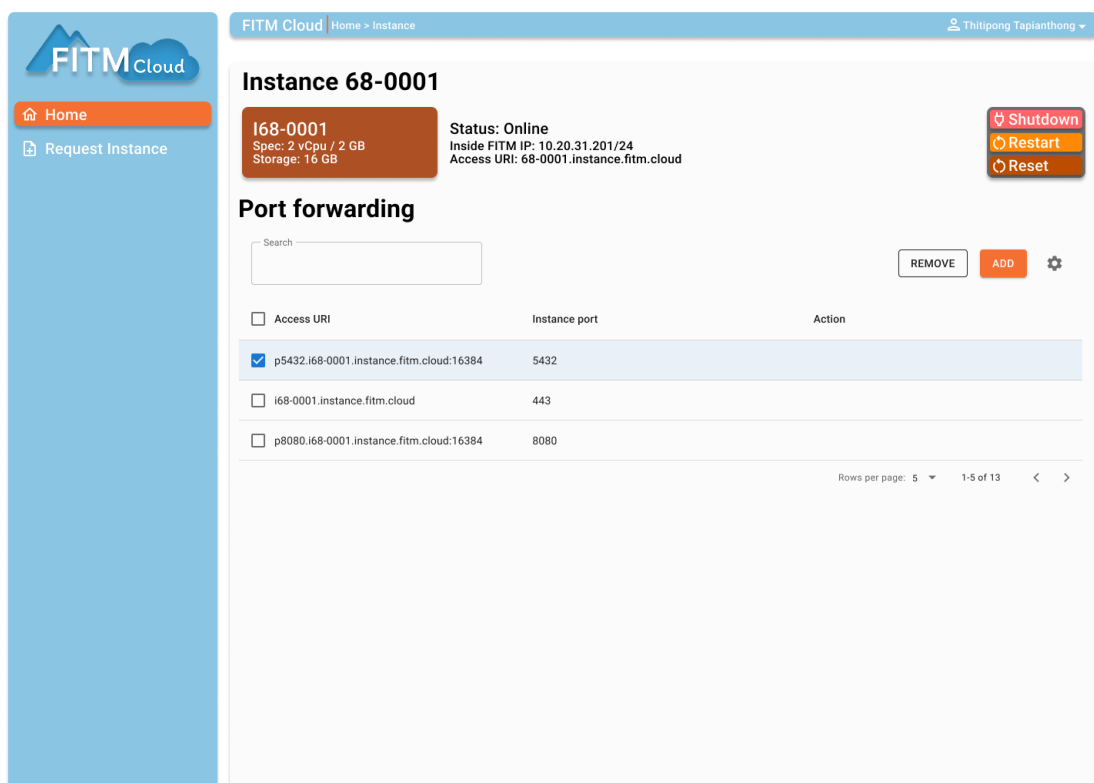
โดยมีการออกแบบหน้าจอหลัก ๆ ดังแสดงในภาพที่ 3-7 – 3-11



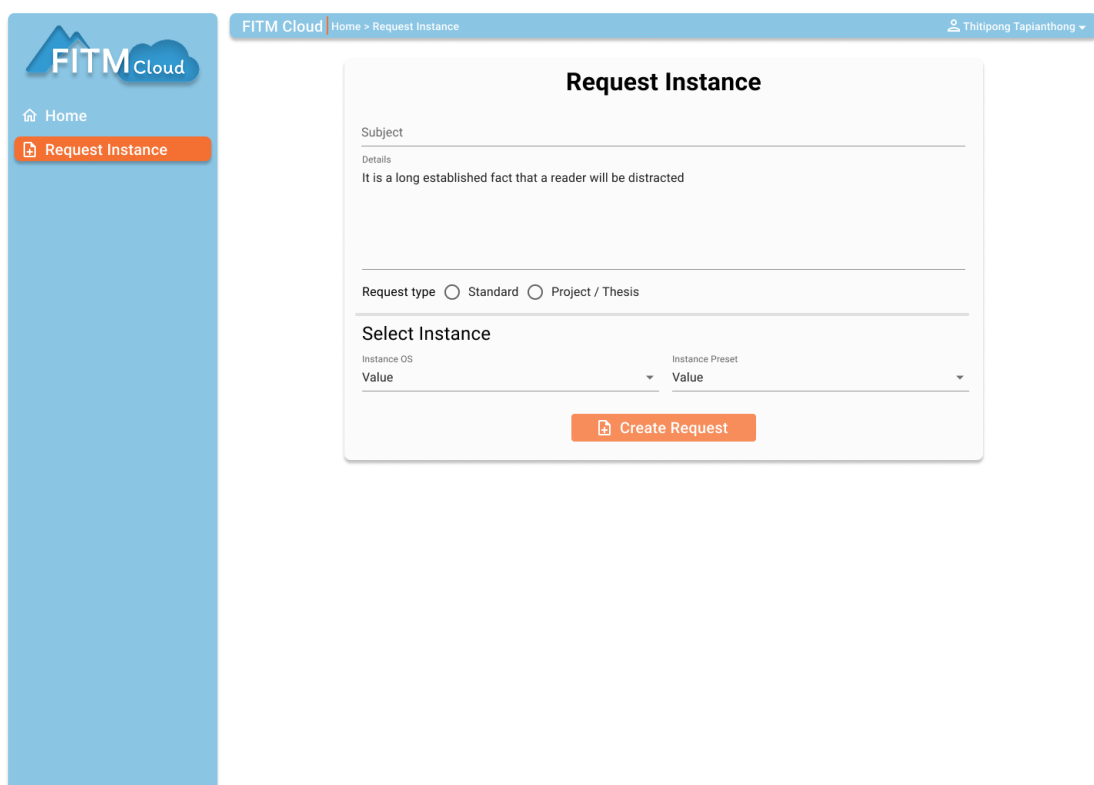
ภาพที่ 3-7 หน้าจอเข้าสู่ระบบ (Login Page)



ภาพที่ 3-8 หน้าจอแดชบอร์ด (Dashboard)



ภาพที่ 3-9 หน้าจอจัดการเครื่องเสมือน (Instance Management)



FITM Cloud | Home > Request Instance | Thitipong Tapianthong

Request Instance

Subject
Details
It is a long established fact that a reader will be distracted

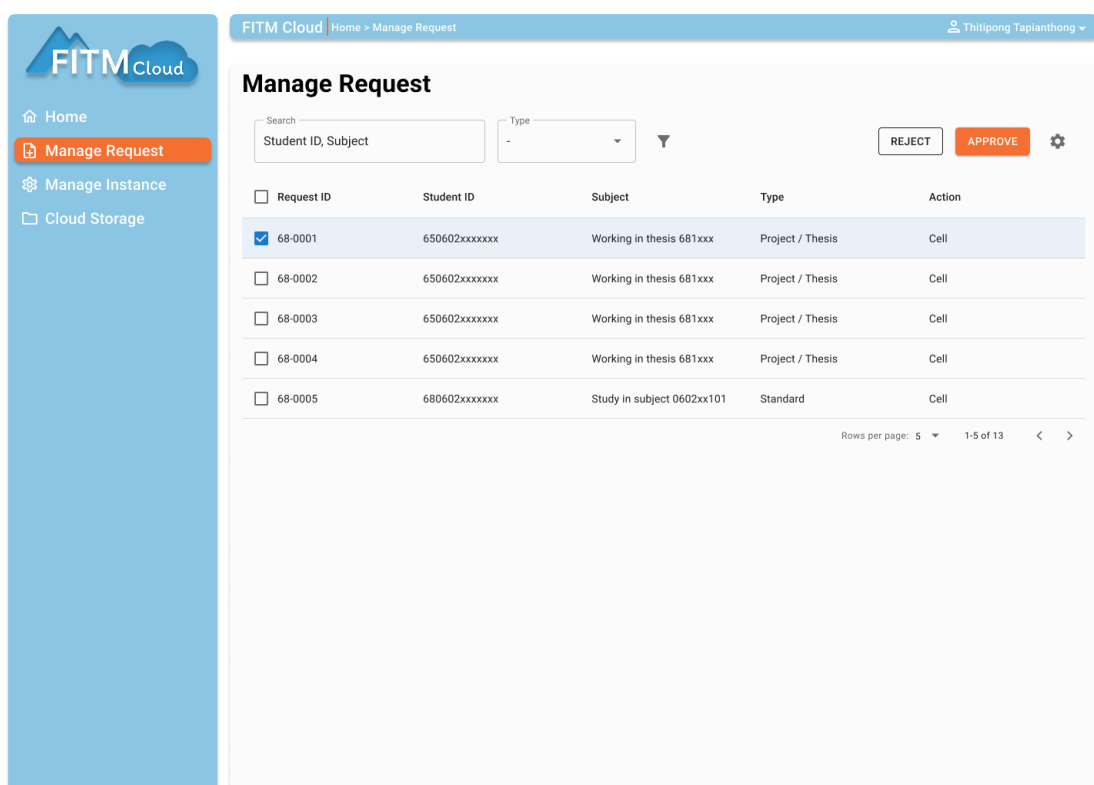
Request type ☐ Standard ☐ Project / Thesis

Select Instance

Instance OS Value Instance Preset Value

[Create Request](#)

ภาพที่ 3-10 หน้าจอคำร้องขอเครื่องเสมือน (Instance Request)



FITM Cloud | Home > Manage Request | Thitipong Tapianthong

Manage Request

Search: Student ID, Subject Type: - Filter: [X] REJECT APPROVE [Settings]

☐ Request ID	Student ID	Subject	Type	Action
☒ 68-0001	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0002	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0003	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0004	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0005	680602xxxxxx	Study in subject 0602xx101	Standard	Cell

Rows per page: 5 1-5 of 13 < >

ภาพที่ 3-11 หน้าจอจัดการคำร้องขอ (Request Management)

3.1.3 การออกแบบฐานข้อมูล

ระบบใช้ PostgreSQL เป็นฐานข้อมูลหลัก โดยออกแบบตารางหลักดังนี้

- **User** เก็บข้อมูลผู้ใช้งาน การยืนยันตัวตน และความสัมพันธ์กับเซสชัน/บัญชี
- **Session** เก็บข้อมูลเซสชันการเข้าสู่ระบบของผู้ใช้
- **Account** เก็บข้อมูลบัญชีผู้ใช้จากผู้ให้บริการภายนอก (เช่น OAuth)
- **Verification** เก็บข้อมูลสำหรับการยืนยัน (เช่น โทเค็น/อีเมล)
- **staff_list** รายชื่อบุคลากร/อาจารย์ที่ได้รับอนุญาต
- **user_public_key** คีย์สาธารณะของผู้ใช้ (เช่น SSH public key)
- **network** ข้อมูลเครือข่าย ชื่อเครือข่าย และเกตเวย์
- **ip_address** รายการที่อยู่ IP และการผูกกับเครื่องเสมือน
- **semester** ข้อมูลภาคการศึกษา ช่วงเวลา และสถานะการใช้งาน
- **pve_node** โหนดของ Proxmox VE และสถานะของโหนด
- **instance_template** แม่แบบ VM/LXC และโหนดต้นทางที่ใช้สร้างเครื่อง
- **instance_course** ข้อมูลรายวิชาและผู้รับผิดชอบสำหรับการจัดสรรเครื่อง
- **instance_request** คำร้องขอเครื่องเสมือนของผู้ใช้ พร้อมสเปกที่ต้องการ
- **instance** เครื่องเสมือนที่ถูกจัดสรรให้ผู้ใช้ รวมสถานะการทำงานและโหนดที่รัน
- **instance_request_extends** คำร้องขอเพิ่มเติม/ขยายสำหรับเครื่องที่มีอยู่

3.1.4 การออกแบบ API และการสื่อสาร

ระบบใช้ RESTful API เป็นหลักในการสื่อสารระหว่างส่วนประกอบ โดยมีการออกแบบดังนี้

- **Authentication Endpoints** สำหรับ Login, Logout และ Token Management
- **Instance Management Endpoints** สำหรับการจัดการเครื่องเสมือน (CRUD Operations)

3.1.5 การออกแบบระบบ Message Queue

รองรับการประมวลผลแบบ Asynchronous โดยใช้ RabbitMQ เป็น Message Broker ออกแบบคิวหลักดังนี้

- **Instance Creation Queue** สำหรับงานการสร้างเครื่องเสมือน
- **Instance Deletion Queue** สำหรับงานการลบเครื่องเสมือน
- **Instance Update Queue** สำหรับงานอัปเดตสถานะของเครื่องเสมือน

ตารางที่ 3-1 สรุปตาราง คีย์หลัก และคีย์ต่างประเทศจาก Prisma Schema

ชื่อตาราง	คีย์หลัก (PK)	คีย์ต่างประเทศ (FK)	รายละเอียด
user	id (String)	—	ผู้ใช้ และ ข้อมูล ยืนยัน ตัว ตน (อีเมลไม่ซ้ำ)
session	id (String)	userId -> user(id)	เซสชันผู้ใช้, token ไม่ซ้ำ
account	id (String)	userId -> user(id)	บัญชีจากผู้ให้บริการภายนอก
verification	id (String)	—	ข้อมูลการยืนยัน (เช่น โทเค็น)
staff_list	id (Int)	—	รายชื่อบุคลากร, email ไม่ซ้ำ
user_public_key	id (Int)	user_id -> user(id)	คีย์ สาธารณะ ของ ผู้ใช้ (SSH key)
network	id (Int)	—	ข้อมูลเครือข่าย / เกตเวย์
ip_address	id (Int)	network_id -> network(id); instance_id -> instance(id) (ตัว เลือก)	IP address ผูกกับเครือข่าย/อินสแตนซ์ (1-1 ตัวเลือก)
semester	id (Int)	—	ภาคการศึกษา, name ไม่ซ้ำ
pve_node	id (Int)	—	โหนด Proxmox VE, name ไม่ซ้ำ
instance_template	id (Int)	vm_template_host -> pve_node(name)	แม่แบบ VMLXC สำหรับสร้างเครื่อง
instance_course	id (Int)	—	รายวิชา/ผู้รับผิดชอบ
instance_request	id (Int)	user_id -> user(id); course_id -> instance_course(id); template_id -> instance_template(id)	คำร้องขอเครื่องเสมือน (สเปก/สถานะ)
instance	id (Int)	user_id -> user(id); course_id -> instance_course(id) (ตัว เลือก); semester_id -> semester(id) (ตัว เลือก); template_id -> instance_template(id); pve_node -> pve_node(name)	เครื่องเสมือนที่จัดสรรให้ผู้ใช้
instance_request_extends	id (Int)	instance_id -> instance(id); instance_requestId -> instance_request(id) (ตัวเลือก)	คำร้องขอเพิ่มเติมสำหรับเครื่อง

ตารางที่ 3-2 รายการ Enum และค่าที่อนุญาตในสคีมาฐานข้อมูล

ชื่อ Enum	ค่า
instance_state	active, deleted, archived
instance_status	pending, running, stopped
instance_request_type	course, project, personal
request_state	pending, approved, rejected, cancelled
vm_type	qemu, lxc
node_status	online, offline, maintenance, unknown

ตารางที่ 3-3 รายละเอียดฟิลด์ของตาราง user

ตาราง: user			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	String	PK	รหัสผู้ใช้ (ไอดีหลัก)
name	String	—	ชื่อผู้ใช้
email	String	UNIQUE	อีเมล (ไม่ซ้ำ)
emailVerified	Boolean	DEFAULT false	สถานะยืนยันอีเมล
image	String?	NULLABLE	URL รูปภาพโปรไฟล์
createdAt	DateTime	DEFAULT now()	วันที่สร้าง
updatedAt	DateTime	DEFAULT now(), UPDATED_AT	วันที่อัปเดตล่าสุด
sessions	Session[]	RELATION (virtual)	ความสัมพันธ์ 1-หลาย ไปยัง session
accounts	Account[]	RELATION (virtual)	ความสัมพันธ์ 1-หลาย ไปยัง account
user_public_key	user_public_key[]	RELATION (virtual)	คีย์สาธารณะของผู้ใช้
instance_request	instance_request[]	RELATION (virtual)	คำร้องขอเครื่องของผู้ใช้
instance	instance[]	RELATION (virtual)	เครื่องเสมือนของผู้ใช้

ตารางที่ 3-4 รายละเอียดฟิลด์ของตาราง session

ตาราง: session			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	String	PK	ไอดีเซสชัน
expiresAt	DateTime	—	วันหมดอายุ
token	String	UNIQUE	โทเค็นเซสชัน
createdAt	DateTime	DEFAULT now()	วันที่สร้าง
updatedAt	DateTime	UPDATED_AT	วันที่อัปเดต
ipAddress	String?	NULLABLE	ไอพีจากผู้ใช้
userAgent	String?	NULLABLE	ข้อมูลเบราว์เซอร์/เอเจนต์
userId	String	FK -> user(id), ON DELETE CASCADE	อ้างอิงผู้ใช้

ตารางที่ 3-5 รายละเอียดฟิลด์ของตาราง account

ตาราง: account			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	String	PK	ไอดีแอดเคานต์
accountId	String	—	ไอดีบัญชีจากผู้ให้บริการ
providerId	String	—	ผู้ ให้ บริการ (เช่น github, google)
userId	String	FK -> user(id), ON DELETE CASCADE	อ้างอิงผู้ใช้
accessToken	String?	NULLABLE	โทเค็นการเข้าถึง
refreshToken	String?	NULLABLE	โทเค็นรีเฟรช
idToken	String?	NULLABLE	ไอดีโทเค็น (OIDC)
accessTokenExpiresAt	DateTime?	NULLABLE	วันหมดอายุ access token
refreshTokenExpiresAt	DateTime?	NULLABLE	วันหมดอายุ refresh token
scope	String?	NULLABLE	สโคปการเข้าถึง
password	String?	NULLABLE	รหัสผ่าน (กรณี local)
createdAt	DateTime	DEFAULT now()	วันที่สร้าง
updatedAt	DateTime	UPDATED_AT	วันที่อัปเดต

ตารางที่ 3-6 รายละเอียดฟิลด์ของตาราง verification

ตาราง: verification			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	String	PK	ไอดีรายการยืนยัน
identifier	String	—	ตัวระบุ
value	String	—	ค่า (เช่น โทเค็น)
expiresAt	DateTime	—	วันหมดอายุ
createdAt	DateTime	DEFAULT now()	วันที่สร้าง
updatedAt	DateTime	DEFAULT now(), UPDATED_AT	วันที่อัปเดต

ตารางที่ 3-7 รายละเอียดฟิลด์ของตาราง staff_list

ตาราง: staff_list			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีรายการ
email	String	UNIQUE	อีเมลบุคลากร
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-8 รายละเอียดฟิลด์ของตาราง user_public_key

ตาราง: user_public_key			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดี
user_id	String	FK -> user(id), ON DELETE CASCADE	อ้างอิงผู้ใช้
public_key	String	—	คีย์สาธารณะ
created_at	DateTime	DEFAULT now()	วันที่สร้าง
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-9 รายละเอียดฟิลด์ของตาราง network

ตาราง: network			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีเครือข่าย
name	String	—	ชื่อเครือข่าย
network	String	—	ช่วงเครือข่าย (CIDR)
gateway	String	—	เกตเวย์
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-10 รายละเอียดฟิลด์ของตาราง ip_address

ตาราง: ip_address			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดี
network_id	Int	FK -> network(id), ON DELETE CASCADE	อ้างอิงเครือข่าย
ip	String	UNIQUE	ที่อยู่ไอพี
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)
instance_id	Int?	UNIQUE, NULLABLE, FK -> instance(id), ON DELETE SET NULL	ผูกกับเครื่อง (1-1 ตัวเลือก)

ตารางที่ 3-11 รายละเอียดฟิลด์ของตาราง semester

ตาราง: semester			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีภาคการศึกษา
name	String	UNIQUE	ชื่อภาคการศึกษา
start_at	DateTime	—	วันเริ่มต้น
end_at	DateTime	—	วันสิ้นสุด
active	Boolean	DEFAULT false	เปิดใช้งาน
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-12 รายละเอียดฟิลด์ของตาราง pve_node

ตาราง: pve_node			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีโหนด
name	String	UNIQUE	ชื่อโหนด
status	node_status	—	สถานะโหนด
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-13 รายละเอียดฟิลด์ของตาราง instance_template

ตาราง: instance_template			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีแม่แบบ
os_name	String	—	ชื่อระบบปฏิบัติการ
vm_template_id	String	—	ไอดีแม่แบบใน PVE
vm_template_host	String	FK -> pve_node(name)	โฮสต์แม่แบบ (อ้างอิงชื่อโหนด)
vm_type	vm_type	—	ประเภท VM/LXC
based_size	Int	—	ขนาดดิสก์พื้นฐาน (GB)
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-14 รายละเอียดฟิลด์ของตาราง instance_course

ตาราง: instance_course			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีรายวิชา
course_id	String	—	รหัสรายวิชา
course_title	String	—	ชื่อรายวิชา
main_staff	Int	—	ผู้รับผิดชอบหลัก
assistant_staff_1	Int?	NULLABLE	ผู้ช่วย 1
assistant_staff_2	Int?	NULLABLE	ผู้ช่วย 2
assistant_staff_3	Int?	NULLABLE	ผู้ช่วย 3
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-15 รายละเอียดฟิลด์ของตาราง instance_request

ตาราง: instance_request			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีคำร้อง
user_id	String	FK -> user(id), ON DELETE CASCADE	ผู้ร้องขอ
title	String	—	ชื่อคำร้อง/โครงการ
hostname	String	—	ชื่อโฮสต์ที่ต้องการ
description	String	—	รายละเอียด
type	instance_request_type	—	ประเภทคำร้อง (course/project/personal)
course_id	Int	FK -> instance_course(id), ON DELETE CASCADE	รายวิชา
template_id	Int	FK -> instance_template(id), ON DELETE CASCADE	แม่แบบที่จะใช้
cpus	Int	—	จำนวนคอร์ CPU
memory	Int	—	หน่วยความจำ (MB)
disk	Int	—	ขนาดดิสก์ (GB)
state	request_state	DEFAULT pending	สถานะคำร้อง
reason	String?	NULLABLE	เหตุผลประกอบ
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต

ตารางที่ 3-16 รายละเอียดฟิลด์ของตาราง instance

ตาราง: instance			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีเครื่องเสมือน
user_id	String	FK -> user(id), ON DELETE CASCADE	เจ้าของเครื่อง
title	String	—	ชื่อเครื่อง
hostname	String	—	โฮสต์เนม
description	String	—	รายละเอียด
type	instance_request_type	—	ประเภทการใช้งาน
course_id	Int	FK -> instance_course(id), ON DELETE CASCADE, NULLABLE	รายวิชา (ถ้ามี)
semester_id	Int?	FK -> semester(id), ON DELETE CASCADE, NULLABLE	ภาคการศึกษา (ถ้ามี)
template_id	Int	FK -> instance_template(id), ON DELETE CASCADE	แม่แบบ
cpus	Int	—	คอร์ CPU
memory	Int	—	หน่วยความจำ (MB)
disk	Int	—	ขนาดดิสก์ (GB)
state	instance_state	DEFAULT active	สถานะทรัพยากร
status	instance_status	DEFAULT pending	สถานะการทำงาน
pve_node	String	FK -> pve_node(name), ON DELETE CASCADE	ชื่อโหนด PVE ที่รัน
vm_id	Int	—	ไอดี VM ใน PVE
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
deleted_at	DateTime?	NULLABLE	วันที่ลบ (เชิงตรรกะ)

ตารางที่ 3-17 รายละเอียดฟิลด์ของตาราง instance_request_extends

ตาราง: instance_request_extends			
ฟิลด์	ประเภทข้อมูล	ข้อจำกัด	รายละเอียด
id	Int	PK, AUTO INCREMENT	เลขไอดีรายการ
instance_id	Int	FK -> instance(id), ON DELETE CASCADE	เครื่องที่เกี่ยวข้อง
title	String	—	ชื่อคำร้องเพิ่มเติม
description	String	—	รายละเอียด
state	request_state	DEFAULT pending	สถานะคำร้อง
reason	String?	NULLABLE	เหตุผลประกอบ
created_at	DateTime	DEFAULT now()	วันที่สร้าง
updated_at	DateTime	DEFAULT now()	วันที่อัปเดต
instance_requestid	Int?	FK -> instance_request(id), NULLABLE	อ้างอิงคำร้องหลัก (ถ้ามี)

3.2 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

3.2.1 เทคโนโลยี Frontend

Next.js 14 เป็น React Framework ที่เลือกใช้เนื่องจากคุณสมบัติดังนี้

- **App Router** สำหรับการจัดการ Routing แบบใหม่
- **Built-in TypeScript Support** รองรับ Type Safety

Tailwind CSS ใช้สำหรับการจัดการ Styling เนื่องจาก

- **Utility-first** เขียน CSS ได้เร็วและมีประสิทธิภาพ
- **Responsive Design** รองรับการแสดงผลบนอุปกรณ์ต่าง ๆ

3.2.2 เทคโนโลยี Backend

Elysia.js เป็น Web Framework ที่ใช้กับ Bun Runtime โดยมีเหตุผลในการเลือกใช้ดังนี้

- **High Performance** มีประสิทธิภาพสูงกว่า Node.js Framework ทั่วไป
- **Type Safety** รองรับ TypeScript แบบเต็มรูปแบบ
- **OpenAPI Integration** สร้าง API Documentation อัตโนมัติ

Prisma ORM ใช้สำหรับการจัดการฐานข้อมูล เนื่องจาก

- **Type-safe Database Client** ป้องกันข้อผิดพลาดในการเขียน Query
- **Database Migrations** จัดการการเปลี่ยนแปลง Schema อย่างเป็นระบบ
- **Database Introspection** สร้าง Schema จากฐานข้อมูลที่มีอยู่

3.2.3 Infrastructure และ DevOps

Docker ใช้สำหรับการ Containerization ของแอปพลิเคชัน เพื่อ

- **Environment Consistency** สภาพแวดล้อมการพัฒนาและ Production เหมือนกัน
- **Easy Deployment** ง่ายต่อการ Deploy และ Scale
- **Resource Isolation** แยกทรัพยากรระหว่างแอปพลิเคชัน

Docker Compose ใช้สำหรับการจัดการ Multi-container Application

- **Service Orchestration** จัดการการเริ่มต้นและหยุดทำงานของ Service
- **Network Management** จัดการเครือข่ายระหว่าง Container
- **Volume Management** จัดการการแชร์ข้อมูลระหว่าง Container

3.2.4 เครื่องมือการพัฒนา

Version Control และ Collaboration

- **Git** สำหรับการควบคุมเวอร์ชันของโค้ด
- **GitHub** สำหรับการเก็บโค้ดและการทำงานร่วมกัน
- **GitHub Actions** สำหรับ CI/CD Pipeline

Code Quality และ Testing

- **ESLint และ Prettier** สำหรับการตรวจสอบและจัดรูปแบบโค้ด
- **TypeScript** สำหรับ Static Type Checking

บทที่ 4

ผลการดำเนินงาน

ในการพัฒนาระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ ได้ดำเนินการออกแบบและพัฒนาระบบที่ประกอบด้วย 3 ส่วนหลัก คือ ส่วนติดต่อผู้ใช้ (Frontend) ส่วนบริการ API (API Service) และส่วนจัดการเครื่องเสมือน (VM Operations Service) พร้อมทั้งระบบฐานข้อมูล ระบบคิวข้อความ และระบบติดตามการทำงาน ในส่วนของผลการดำเนินงานที่ได้จะประกอบด้วยโครงสร้างโปรเจกต์ การจัดการไฟล์และโฟลเดอร์ การกำหนดค่าระบบ และการจัดการแพ็กเกจต่าง ๆ ทั้งนี้ ระบบที่พัฒนาขึ้นได้ถูกออกแบบให้รองรับการทำงานแบบ Microservices Architecture เพื่อความยืดหยุ่นในการขยายระบบและการบำรุงรักษา โดยใช้เทคโนโลยีสมัยใหม่ที่เหมาะสมกับการใช้งานในสภาพแวดล้อมการศึกษา

4.1 โครงสร้างโปรเจกต์หลัก

จากการออกแบบและพัฒนาระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ ระบบประกอบไปด้วย 3 โฟลเดอร์หลัก ได้แก่

4.1.1 โฟลเดอร์ Apps

โฟลเดอร์ **Apps** เป็นที่เก็บแอปพลิเคชันหลักทั้งหมดของระบบ ซึ่งประกอบด้วย 3 บริการหลัก คือ **Midori**, **Momoi**, และ **Yuzu** ที่ทำงานร่วมกันในรูปแบบ Microservices Architecture

- 1) **Midori** - ส่วนติดต่อผู้ใช้ (Frontend) ที่พัฒนาด้วย Next.js
- 2) **Momoi** - บริการ API หลัก (API Service) ที่พัฒนาด้วย Elysia.js
- 3) **Yuzu** - บริการจัดการเครื่องเสมือน (VM Operations Service)

แต่ละบริการในโฟลเดอร์ **Apps** มีโครงสร้างและหน้าที่เฉพาะตัว โดยการแยกเป็นบริการย่อยจะช่วยให้ระบบมีความยืดหยุ่นในการพัฒนา การทดสอบ และการบำรุงรักษา

4.1.2 โฟลเดอร์ Packages

โฟลเดอร์ **Packages** ใช้สำหรับเก็บแพ็กเกจและไลบรารีที่ใช้ร่วมกันระหว่างบริการต่าง ๆ ในระบบ ภายในโฟลเดอร์นี้จะเก็บโค้ดที่สามารถนำไปใช้ซ้ำได้ (Reusable Code) เพื่อลดการเขียนโค้ดซ้ำซ้อน ตัวอย่างแพ็กเกจที่เก็บ ได้แก่

- 1) **Auth** - จัดการระบบยืนยันตัวตนและการให้สิทธิ์
- 2) **Database** - จัดการฐานข้อมูลและ ORM Models
- 3) **Utils** - ฟังก์ชันและเครื่องมือที่ใช้ร่วมกัน

การจัดระเบียบแพ็คเกจในโฟลเดอร์ **Packages** จะช่วยให้การจัดการโค้ดเป็นระเบียบ สะดวกต่อการบำรุงรักษา และสามารถนำไปใช้ในโปรเจกต์อื่น ๆ ได้ง่าย

4.1.3 โฟลเดอร์ Config

โฟลเดอร์ **Config** ใช้สำหรับเก็บไฟล์การกำหนดค่าต่าง ๆ ของระบบ เช่น การกำหนดค่าบริการ Infrastructure การตั้งค่า Monitoring และการกำหนดค่า Cloud Scripts ที่จำเป็นสำหรับการทำงานของระบบ โดยการแยกไฟล์การกำหนดค่าออกมาเป็นโฟลเดอร์แยกจะช่วยให้ง่ายต่อการจัดการและแก้ไขการตั้งค่าของระบบ โดยไม่ต้องไปแก้ไขโค้ดหลักของแอปพลิเคชัน

4.2 โครงสร้างแอปพลิเคชันและบริการ

4.2.1 ส่วนประกอบหลักของระบบ

นอกจากโฟลเดอร์ทั้งสามที่กล่าวมาแล้ว ในโครงสร้างโปรเจกต์ยังมีไฟล์และโฟลเดอร์สำคัญอื่น ๆ อีกหลายส่วน ได้แก่ ไฟล์ **package.json** ซึ่งเป็นไฟล์หลักที่ใช้ในการจัดการ Monorepo และกำหนด Workspaces สำหรับแต่ละแอปพลิเคชันและแพ็คเกจ ไฟล์นี้จะทำหน้าที่กำหนดการพึ่งพา (Dependencies) ระหว่างแพ็คเกจต่าง ๆ รวมถึงการตั้งค่า TypeScript และ Bun Runtime ที่ใช้ในการพัฒนาและรันแอปพลิเคชัน พร้อมทั้งจัดการ Scripts สำหรับการ Build, Test, และ Deploy ระบบ ทำให้ผู้พัฒนาสามารถจัดการโปรเจกต์ขนาดใหญ่ได้อย่างเป็นระบบและมีประสิทธิภาพ

4.2.2 โครงสร้างระบบทั้งหมด

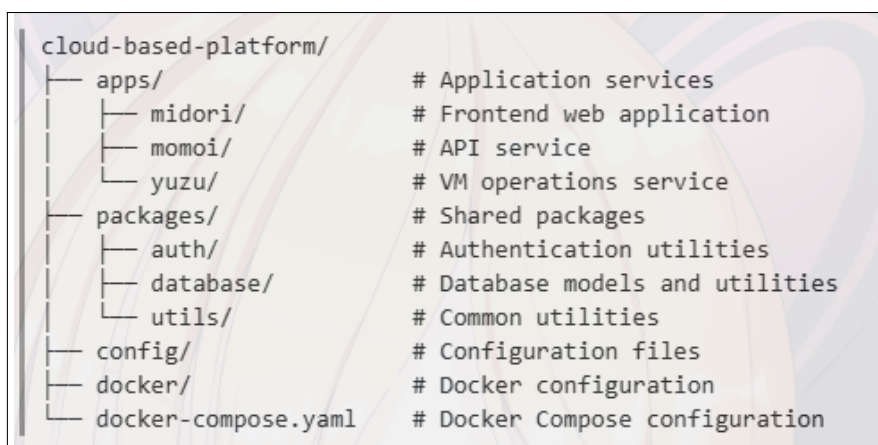
ภาพที่ 4-1 แสดงโครงสร้างโปรเจกต์ทั้งหมดของระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ จะเห็นว่ามีการจัดระเบียบไฟล์และโฟลเดอร์อย่างเป็นระบบ เพื่อความสะดวกในการพัฒนาและบำรุงรักษา โดยโครงสร้างประกอบด้วยโฟลเดอร์หลัก 3 โฟลเดอร์ ได้แก่ Apps, Packages และ Config ซึ่งทำหน้าที่จัดการแอปพลิเคชัน แพ็คเกจรวม และการกำหนดค่าตามลำดับ อีกทั้งยังมีโฟลเดอร์เสริมอื่น ๆ เช่น Docker และ Docs ดังที่กล่าวไว้ข้างต้น

นอกจากนี้ยังประกอบด้วยไฟล์สำคัญอีกหลายไฟล์ ได้แก่

- docker-compose.yaml สำหรับการจัดการ Infrastructure Services
- tsconfig.json สำหรับการกำหนดค่า TypeScript ทั้งโปรเจกต์
- bun.lock สำหรับล็อกเวอร์ชันของแพ็คเกจที่ใช้

- .env.example สำหรับตัวอย่างการตั้งค่าตัวแปรสภาพแวดล้อม
- setup.sh และ setup.ps1 สำหรับสคริปต์การติดตั้งเริ่มต้น
- ไฟล์เอกสารต่าง ๆ เช่น README.md และ TODO.md

การแบ่งโครงสร้างเป็นส่วนย่อยเช่นนี้ช่วยให้การจัดการโปรเจกต์มีความเป็นระเบียบ และง่ายต่อการทำงานร่วมกันเป็นทีม เมื่อต้องการแก้ไขหรือเพิ่มฟีเจอร์ในบริการใดบริการหนึ่งก็สามารถเข้าไปทำงานได้โดยตรงในโฟลเดอร์นั้น ๆ โดยไม่กระทบกับส่วนอื่น ๆ ของระบบ



ภาพที่ 4-1 โครงสร้างโปรเจกต์ทั้งหมด

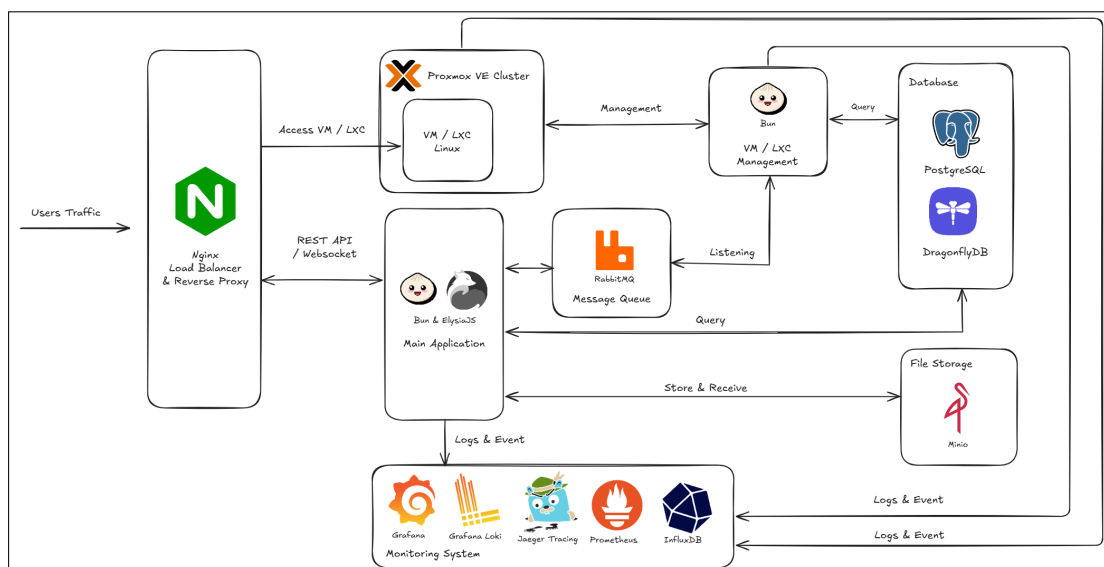
การจัดโครงสร้างโปรเจกต์ในลักษณะ Monorepo นี้ไม่เพียงแต่ช่วยให้โปรเจกต์เป็นระเบียบ แต่ยังช่วยลดความซับซ้อนในการจัดการ Dependencies ระหว่างแพ็คเกจ ทำให้การพัฒนาและการ Deploy เป็นทีมง่ายขึ้น และสามารถแชร์โค้ดระหว่างบริการต่าง ๆ ได้อย่างมีประสิทธิภาพ

4.3 สถาปัตยกรรมระบบและเทคโนโลยี

4.3.1 การออกแบบสถาปัตยกรรมแบบ Microservices

ระบบแพลตฟอร์มคลาวด์ที่พัฒนาขึ้นใช้สถาปัตยกรรมแบบ Microservices ซึ่งประกอบด้วยบริการหลัก 3 ตัว ที่ทำงานอิสระแต่สื่อสารกันผ่าน API และ Message Queue โดยแต่ละบริการมีหน้าที่และความรับผิดชอบที่แตกต่างกัน เพื่อให้ระบบมีความยืดหยุ่นและสามารถขยายตัวได้ง่าย

ภาพที่ 4-2 แสดงสถาปัตยกรรมระบบโดยรวม โดยมี **Midori** เป็นส่วนติดต่อผู้ใช้ที่พัฒนาด้วย Next.js และ Material UI **Momoi** เป็นบริการ API หลักที่จัดการการยืนยันตัวตน การจัดการผู้ใช้ และการเชื่อมต่อกับฐานข้อมูล และ **Yuzu** เป็นบริการจัดการเครื่องเสมือนที่เชื่อมต่อกับ Proxmox VE ผ่าน RabbitMQ



ภาพที่ 4-2 สถาปัตยกรรมระบบโดยรวม

4.3.2 เทคโนโลยีที่ใช้ในการพัฒนา

ในการพัฒนาระบบแพลตฟอร์มคลาวด์นี้ ได้เลือกใช้เทคโนโลยีที่ทันสมัยและเหมาะสมกับการใช้งานในสภาพแวดล้อมการศึกษา โดยเทคโนโลยีหลักที่ใช้ประกอบด้วย

- **Bun Runtime** - JavaScript/TypeScript Runtime ที่มีประสิทธิภาพสูง
- **Next.js** - React Framework สำหรับการพัฒนา Frontend
- **Elysia.js** - Web Framework สำหรับการพัฒนา API
- **PostgreSQL** - ระบบจัดการฐานข้อมูลเชิงสัมพันธ์
- **Prisma ORM** - Object-Relational Mapping สำหรับการจัดการฐานข้อมูล
- **RabbitMQ** - Message Queue สำหรับการสื่อสารระหว่างบริการ
- **Docker** - Container Platform สำหรับการ Deploy และการจัดการ Infrastructure

การเลือกใช้เทคโนโลยีเหล่านี้มีเหตุผลสำคัญ คือ ความเสถียร ประสิทธิภาพ และการรองรับการขยายระบบในอนาคต รวมถึงการมี Community ที่ใหญ่และเอกสารประกอบที่ครบถ้วน ทำให้การพัฒนาและบำรุงรักษาระบบเป็นไปอย่างราบรื่น

4.4 เอกสารประกอบการใช้งาน API และส่วนติดต่อผู้ใช้

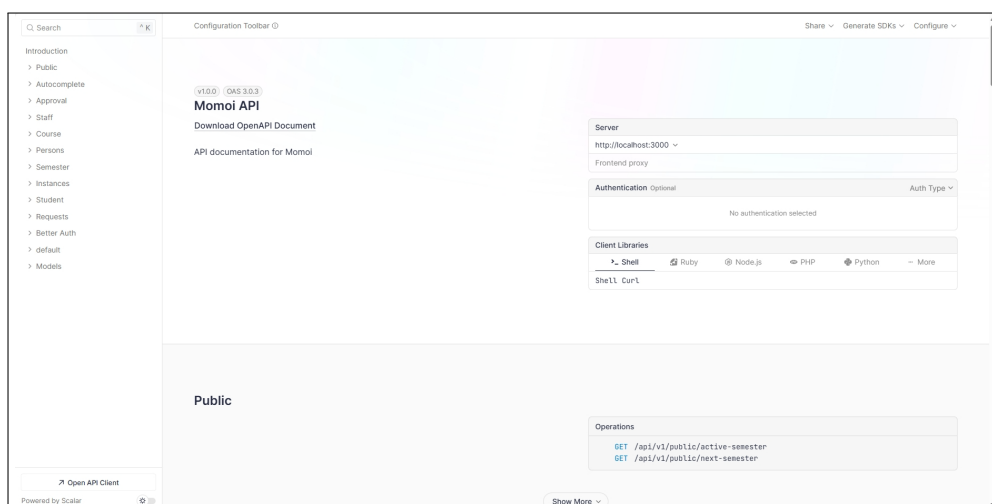
4.4.1 เอกสาร API Documentation

ระบบแพลตฟอร์มคลาวด์ได้จัดทำเอกสาร API Documentation อย่างครบถ้วนโดยใช้ OpenAPI Specification (Swagger) เพื่อให้ผู้พัฒนาและผู้ใช้งานสามารถเข้าใจและใช้งาน API ได้อย่างง่ายดาย

เอกสารนี้ประกอบด้วยรายละเอียดของ Endpoints ต่าง ๆ พารามิเตอร์ที่ต้องการ รูปแบบข้อมูลที่ส่งและรับ รวมทั้งตัวอย่างการใช้งานที่ชัดเจน

ภาพที่ 4-3 แสดงหน้าเอกสาร API Documentation ที่สร้างขึ้นโดยอัตโนมัติจากโค้ด TypeScript ผ่าน Elysia.js และ OpenAPI Plugin ซึ่งมีการจัดหมวดหมู่ API เป็นกลุ่มต่าง ๆ ดังนี้

- **Public API** - สำหรับการใช้งานทั่วไปที่ไม่ต้องมีการยืนยันตัวตน
- **Authentication API** - สำหรับการล็อกอิน ล็อกเอาต์ และการจัดการเซสชัน
- **Student API** - สำหรับนักศึกษาในการจัดการเครื่องเสมือนส่วนตัว
- **Staff API** - สำหรับอาจารย์และเจ้าหน้าที่ในการจัดการระบบ
- **Admin API** - สำหรับผู้ดูแลระบบในการจัดการผู้ใช้และทรัพยากร



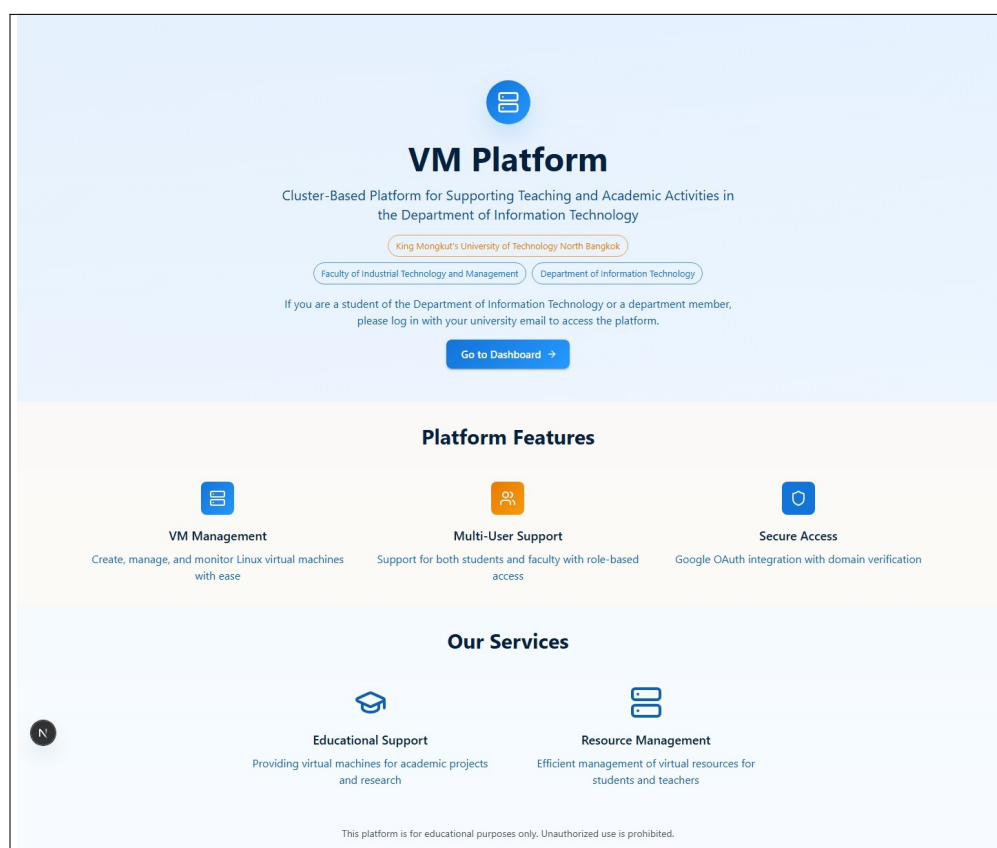
ภาพที่ 4-3 หน้าเอกสาร API Documentation

เอกสาร API นี้มีการอัปเดตอัตโนมัติเมื่อมีการเปลี่ยนแปลงโค้ด และสามารถทดสอบ API ได้โดยตรงผ่านหน้าเว็บ ทำให้ผู้พัฒนาสามารถทำความเข้าใจและใช้งาน API ได้อย่างรวดเร็วและถูกต้อง

4.4.2 ส่วนติดต่อผู้ใช้ (User Interface)

ส่วนติดต่อผู้ใช้ของระบบได้รับการออกแบบให้ใช้งานง่าย สวยงาม และตอบสนองได้ดีบนอุปกรณ์ต่าง ๆ โดยใช้ Material UI และ Tailwind CSS ในการจัดรูปแบบ ระบบประกอบด้วยหน้าหลักต่าง ๆ ที่มีการจัดระเบียบข้อมูลและฟังก์ชันการทำงานอย่างชัดเจน

ภาพที่ 4-4 แสดงหน้าแรกของระบบ (Landing Page) ที่ออกแบบให้ดึงดูดความสนใจและสื่อสารจุดประสงค์ของระบบได้ชัดเจน โดยมีการนำเสนอฟีเจอร์หลักของระบบ ข้อมูลการติดต่อ และลิงก์สำหรับการล็อกอินเข้าสู่ระบบ



ภาพที่ 4-4 หน้าแรกของระบบ (Landing Page)

4.4.3 Dashboard และระบบจัดการ

หลังจากที่ผู้ใช้ล็อกอินเข้าสู่ระบบแล้ว ผู้ใช้จะเข้าสู่หน้า Dashboard ที่แสดงข้อมูลสำคัญและฟังก์ชันการทำงานต่าง ๆ ตามบทบาทของผู้ใช้ โดย Dashboard ได้รับการออกแบบให้แสดงข้อมูลอย่างเป็นระบบและเข้าใจง่าย พร้อมทั้งมีการใช้งานที่สะดวกและรวดเร็ว

ส่วนติดต่อผู้ใช้ที่พัฒนาขึ้นมีลักษณะสำคัญดังนี้

- **Responsive Design** - รองรับการแสดงผลบนหน้าจอขนาดต่าง ๆ
- **Intuitive Navigation** - การนำทางที่เข้าใจง่ายและสอดคล้องกับมาตรฐาน UX

การออกแบบส่วนติดต่อผู้ใช้ได้คำนึงถึงประสบการณ์ผู้ใช้ (User Experience) เป็นหลัก โดยมุ่งเน้นให้ผู้ใช้สามารถทำงานได้อย่างมีประสิทธิภาพและลดข้อผิดพลาดในการใช้งาน ทั้งนี้ระบบยังมีการจัดการข้อผิดพลาดและการแจ้งเตือนที่ชัดเจน เพื่อให้ผู้ใช้สามารถแก้ไขปัญหาได้อย่างรวดเร็ว

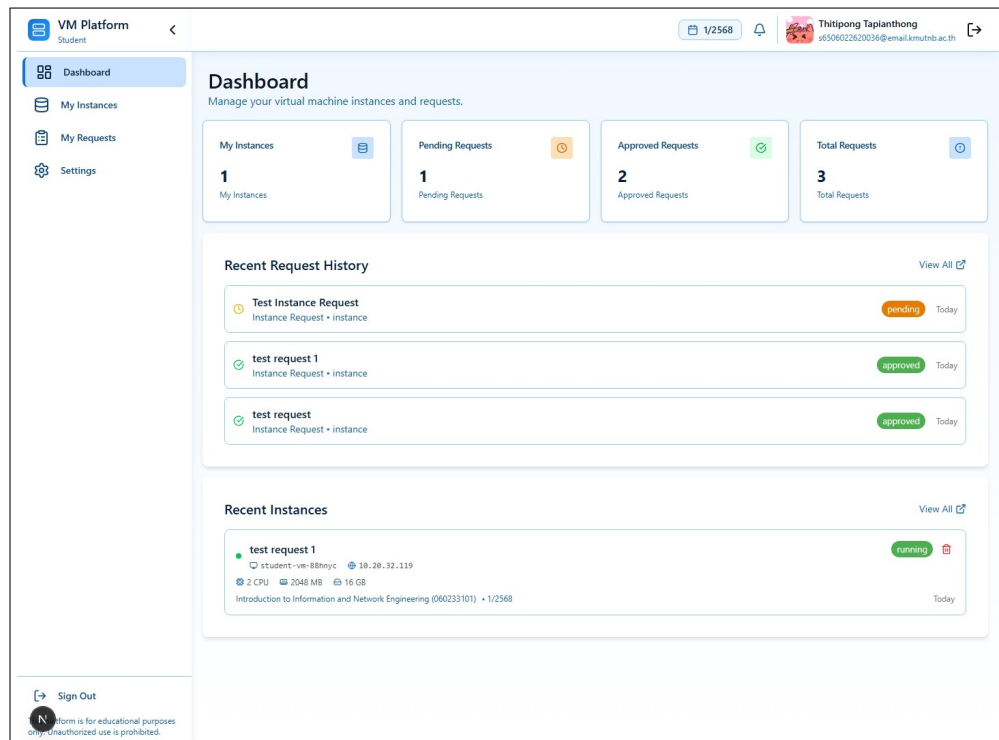
4.5 ตัวอย่างการใช้งานระบบตามบทบาทผู้ใช้

4.5.1 Dashboard สำหรับนักศึกษา

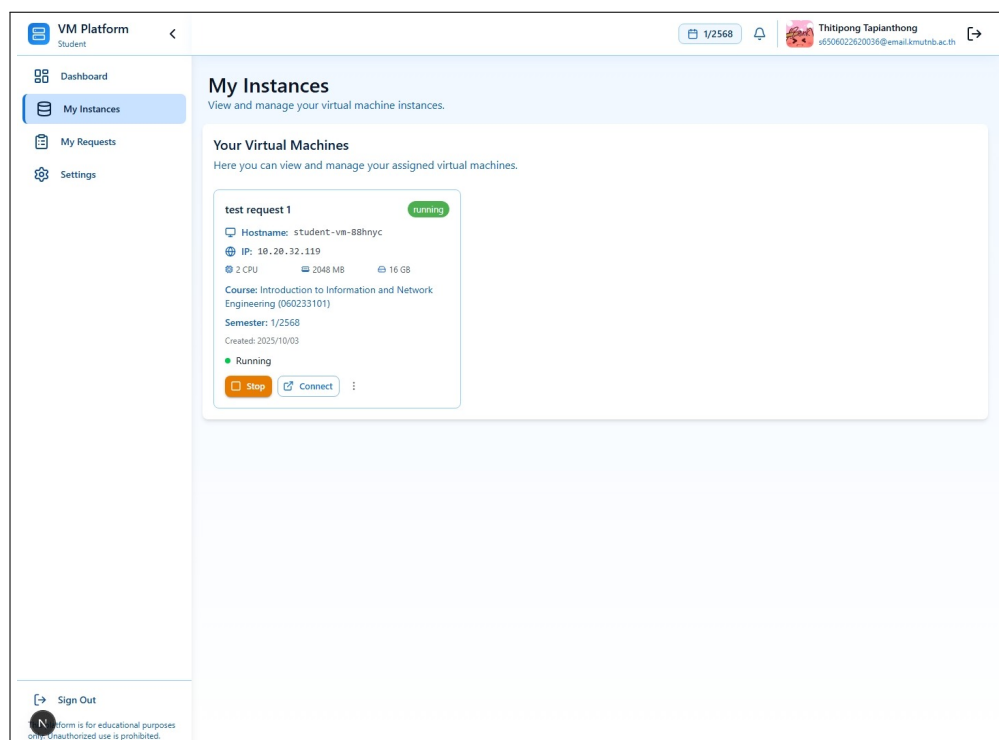
ระบบได้จัดทำ Dashboard เฉพาะสำหรับนักศึกษา เพื่อให้สามารถจัดการเครื่องเสมือนส่วนตัว และติดตามการใช้งานทรัพยากรได้อย่างง่ายดาย ภาพที่ 4-5 แสดง Dashboard หลักของนักศึกษาที่ประกอบด้วยข้อมูลสรุปการใช้งาน สถานะเครื่องเสมือน และฟังก์ชันการจัดการที่สำคัญ

นักศึกษาสามารถดูรายการเครื่องเสมือนทั้งหมดที่ตนเองมีสิทธิ์เข้าถึง พร้อมทั้งสถานะการทำงาน การใช้ทรัพยากร และการจัดการพื้นฐาน ดังแสดงในภาพที่ 4-6 ซึ่งนำเสนอรายละเอียดของแต่ละ Instance อย่างชัดเจน

นอกจากนี้ นักศึกษายังสามารถขอสร้างเครื่องเสมือนใหม่ผ่านฟอร์มที่ออกแบบมาอย่างเรียบง่าย ดังแสดงในภาพที่ 4-7 ซึ่งช่วยให้การขอใช้ทรัพยากรเป็นไปอย่างรวดเร็วและมีประสิทธิภาพ



ภาพที่ 4-5 Dashboard หลักสำหรับนักศึกษา



ภาพที่ 4-6 หน้าจัดการ VM Instances สำหรับนักศึกษา

VM Platform
Student

Dashboard

My Instances

My Requests

Settings

1/2568

Thitipong Tapianthong
#65060126020026@email.kmutnb.ac.th

Request Management

Create new instance requests and manage your existing requests

Create New Request

Submit a request for a new instance or extend an existing one

[New Instance Request](#) [Extend Instance](#)

New Instance Request

Request a new virtual machine instance for your course or project

Title *

Enter request title

Request Type *

Course Work

Hostname *

Enter hostname

Course *

Select a course

Template *

Select a template

Description *

Describe your request requirements...

Resource Specifications

CPU Cores

2

Memory (MB)

2048

Disk (GB)

16

Create Request

Request History

Your submitted requests and their status

Test Instance Request

Instance Request • course • student-vm-22qx47

pending

Today

test request 1

Instance Request • course • student-vm-88hnyx

approved

Today

test request

Instance Request • course • student-vm-egj2pq

approved

Today

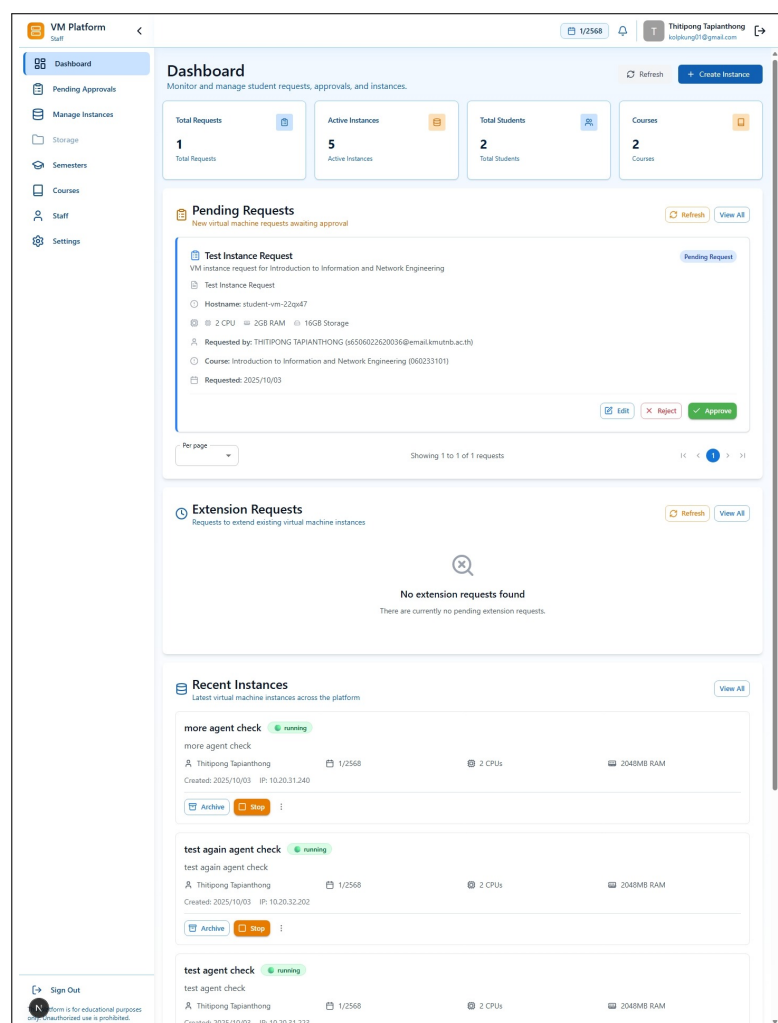
Sign Out

Information is for educational purposes only. Unauthorized use is prohibited.

ภาพที่ 4-7 หน้าจอขอสร้าง VM Instance ใหม่สำหรับนักศึกษา

4.5.2 Dashboard สำหรับอาจารย์และเจ้าหน้าที่

สำหรับอาจารย์และเจ้าหน้าที่ ระบบได้จัดเตรียม Dashboard ที่มีฟังก์ชันการจัดการขั้นสูงมากขึ้น รวมถึงการอนุมัติคำขอ การจัดการหลักสูตร และการตรวจสอบการใช้งานของนักศึกษา ภาพที่ 4-8 แสดง Dashboard หลักสำหรับเจ้าหน้าที่ที่มีข้อมูลสถิติและการควบคุมระบบ



ภาพที่ 4-8 Dashboard หลักสำหรับอาจารย์และเจ้าหน้าที่

อาจารย์และเจ้าหน้าที่สามารถจัดการหลักสูตร เทอมการศึกษา รวมถึงการจัดการรายชื่อของอาจารย์และเจ้าหน้าที่ ดังแสดงในภาพที่ 4-9 - 4-11 ซึ่งช่วยให้สามารถกำหนดทรัพยากรและสิทธิ์การเข้าถึงสำหรับแต่ละวิชาได้อย่างเหมาะสม

นอกจากนี้ ระบบยังมีหน้าสำหรับการอนุมัติคำขอต่าง ๆ จากนักศึกษา ดังแสดงในภาพที่ 4-12 ซึ่งช่วยให้อาจารย์สามารถตรวจสอบและอนุมัติคำขอได้อย่างมีประสิทธิภาพ

VM Platform Staff

1/2568

Thitipong Tapianthong
kolpkung01@gmail.com

Course Management
View and manage course information and staff assignments

Add New Course

Course ID *
e.g., CS101

Course Title *
e.g., Introduction to Computer Science

Main Staff *
Select main staff

Assistant Staff 1
Select assistant

Assistant Staff 2
Select assistant

+ Create Course

Course Lists

Fundamental to Information Technology
Course Code: 060243101
Main: Thitipong Tapianthong
0 requests 6 instances

Introduction to Information and Network Engineering
Course Code: 060233101
Main: Thitipong Tapianthong
3 requests 3 instances

Course Statistics

Total Courses
2

Total Requests
3

Total Instances
9

Sign Out

Platform is for educational purposes only. Unauthorized use is prohibited.

ภาพที่ 4-9 หน้าจัดการหลักสูตรและรายวิชา

VM Platform Staff

1/2568

Thitipong Tapianthong
kolpkung01@gmail.com

Semester Management
Manage academic semesters and set active semester

Active Semester

1/2568
2025/06/23 - 2025/10/31 Active

All Semesters
Manage all academic semesters

+ Add Semester

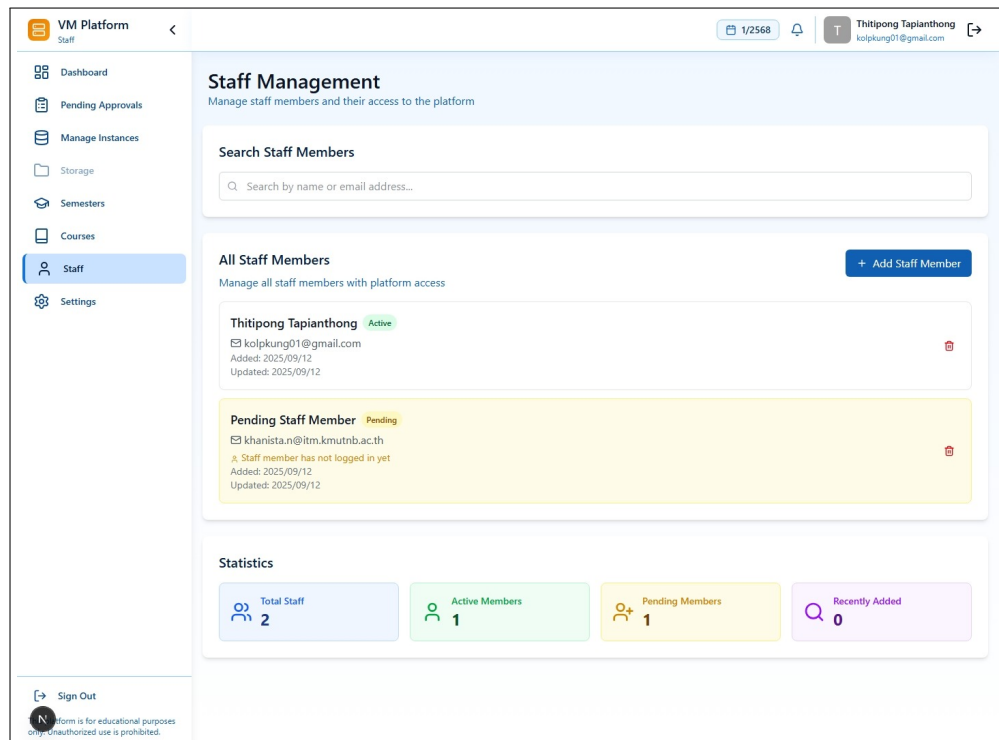
2/2568
2025/11/24 - 2026/03/31
Created: 2025/09/12

1/2568 Active
2025/06/23 - 2025/10/31
Created: 2025/09/12

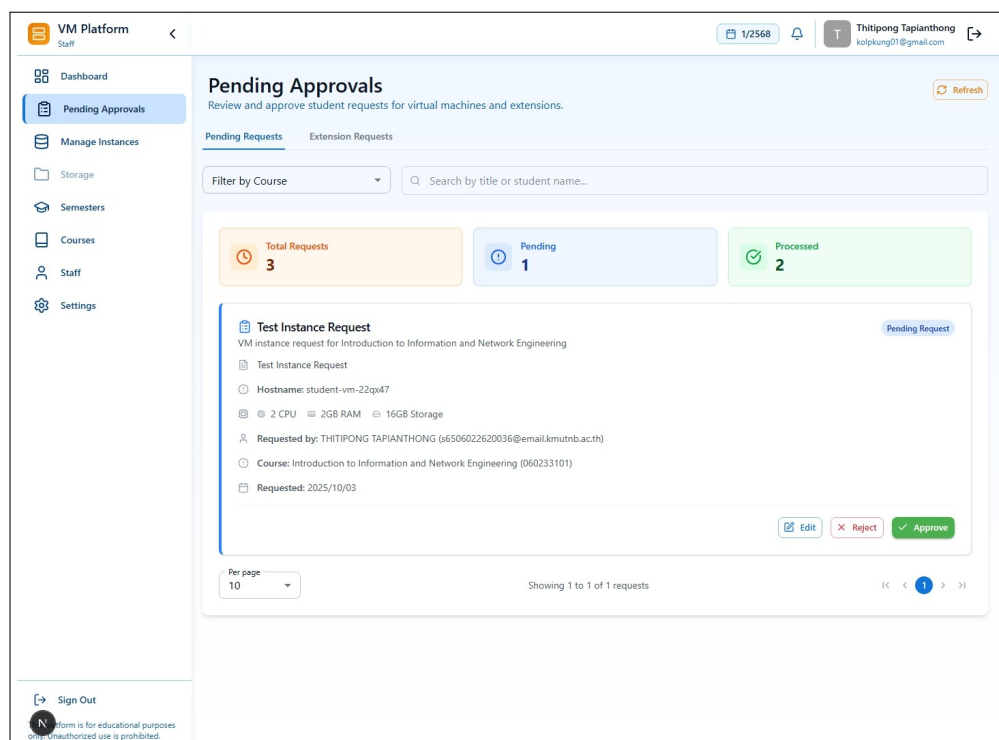
Sign Out

Platform is for educational purposes only. Unauthorized use is prohibited.

ภาพที่ 4-10 หน้าจัดการเทอมการศึกษา



ภาพที่ 4-11 หน้าจัดการรายชื่ออาจารย์



ภาพที่ 4-12 หน้าอนุมัติคำขอที่รอดำเนินการ

4.5.3 ระบบจัดการแบบ Role-Based Access Control

ระบบได้นำเอา Role-Based Access Control (RBAC) มาใช้ในการจัดการสิทธิ์การเข้าถึงข้อมูล และฟังก์ชันต่าง ๆ โดยแบ่งผู้ใช้เป็น 3 กลุ่มหลัก คือ นักศึกษา อาจารย์ และผู้ดูแลระบบ ซึ่งแต่ละกลุ่มจะมีสิทธิ์และหน้าที่ที่แตกต่างกันตามความเหมาะสม

การ ออกแบบ ระบบใน ลักษณะ นี้ช่วยให้การจัดการ ความ ปลอดภัย ของ ข้อมูล เป็นไปอย่างมี ประสิทธิภาพ และผู้ใช้แต่ละกลุ่มจะเห็นเฉพาะข้อมูลและฟังก์ชันที่เกี่ยวข้องกับหน้าที่ของตนเอง ทำให้ การใช้งานง่ายขึ้นและลดความเสี่ยงในการเข้าถึงข้อมูลที่ไม่เหมาะสม

4.6 ผลการทดสอบประสิทธิภาพระบบ

ส่วนนี้สรุปผลการทดสอบประสิทธิภาพของบริการหลักในระบบ โดยแบ่งตามบริการย่อย เพื่อให้ เห็นภาพรวมและเปรียบเทียบสมรรถนะของแต่ละบริการได้อย่างชัดเจน

4.6.1 ผลการทดสอบประสิทธิภาพส่วนติดต่อผู้ใช้ (Midori)

ทำการทดสอบ สมรรถนะ ของ Midori (Frontend/Next.js) โดยมีเป้าหมายการทดสอบคือ <http://localhost:3000> ระยะเวลารวม 20 วินาที และระดับความขนานต่อเส้นทาง 20

ภาพรวมของการทดสอบสะท้อนปริมาณค่าขอรวม อัตราความสำเร็จ 100% และอัตราการส่งผ่านโดยเฉลี่ย ดังตารางที่ 4-1

ตารางที่ 4-1 สรุปผลภาพรวมการทดสอบ Midori

เวลาเริ่ม	ระยะเวลา (s)	Requests	สำเร็จ	ล้มเหลว	Throughput (RPS)
2025-10-18 18:57:00Z	20	14,509	14,509	0	719.62

การทดสอบครอบคลุมเส้นทางหลัก ได้แก่ หน้าแรกและหน้าลงชื่อเข้าใช้ รวมถึง API สาธารณะที่ เรียกผ่าน NextJS Rewrite 2 เส้นทาง สถิติหลักสรุปในตารางที่ 4-2

ตารางที่ 4-2 ผลการทดสอบรายเส้นทาง (Midori)

เส้นทาง	Requests	สำเร็จ	ล้มเหลว	RPS	p50 (ms)	p95 (ms)	p99 (ms)	max (ms)
/	4,164	4,164	0	206.53	53.95	112.46	170.42	291.27
/sign-in	4,198	4,198	0	208.88	55.83	105.50	165.48	293.72
/api/v1/public/active-semester	3,072	3,072	0	152.77	112.56	210.71	280.86	461.16
/api/v1/public/next-semester	3,075	3,075	0	152.96	113.56	209.27	276.64	446.33

ข้อสังเกตและข้อเสนอแนะเบื้องต้น

- ค่าขอทั้งหมดสำเร็จ รวม 720 RPS โดยหน้า UI หลักให้ p50 ราว 54–56 ms ส่วน API ภายใต้อพอร์ท 3000 มี p50 สูงกว่าเนื่องจากผ่านเลเยอร์ Next.js/Proxy
- หากต้องการลด latency ของ API ผ่านพอร์ท 3000 ให้พิจารณาใช้การ proxy แบบ edge/streaming, เปิด compression และ cache ที่เหมาะสมสำหรับ resource คงที่
- ตรวจสอบการทำงานร่วมกับ Momoi ผ่านเครือข่ายภายใน เพื่อลด hop และ revalidation ที่ไม่จำเป็นในเส้นทาง API

4.6.2 ผลการทดสอบประสิทธิภาพบริการ API (Momoi)

เพื่อ ประเมิน สมรรถนะ ของบริการ Momoi (API Service) ได้ดำเนินการ ทดสอบ ภาระ งาน ด้วยการยิงคำขอพร้อมกันต่อเส้นทาง (per-path concurrency) โดยมีเป้าหมายการทดสอบคือ <http://localhost:3001> ที่ระยะเวลารวม 20 วินาที และระดับความขนานต่อเส้นทาง 20

ภาพรวมของการทดสอบสะท้อนปริมาณคำขอรวม อัตราความสำเร็จ 100% และอัตราการส่งผ่านโดยเฉลี่ย ดังตารางที่ 4-3

ตารางที่ 4-3 สรุปผลภาพรวมการทดสอบ Momoi API

เวลาเริ่ม	ระยะเวลา (s)	Requests	สำเร็จ	ล้มเหลว	Throughput (RPS)
2025-10-18 18:53:49Z	20	45,146	45,146	0	2,255.16

จากข้อมูลสรุป ครึ่งนี้คำขอทั้งหมดสำเร็จ (ค่า ok= 45,146) สะท้อนว่าเส้นทางสาธารณะทำงานถูกต้องภายใต้เงื่อนไขการทดสอบดังกล่าวตามข้อมูลในตารางที่ 4-4

ตารางที่ 4-4 ผลการทดสอบรายเส้นทาง (Momoi API)

เส้นทาง	Requests	สำเร็จ	ล้มเหลว	RPS	p50 (ms)	p95 (ms)	p99 (ms)	max (ms)
/api/v1/public/active-semester	22,560	22,560	0	1,126.95	16.01	28.58	37.05	129.53
/api/v1/public/next-semester	22,586	22,586	0	1,131.59	15.94	28.53	37.45	129.62

ข้อสังเกตและข้อเสนอแนะเบื้องต้น

- ค่าขอทั้งหมดสำเร็จภายใต้โหลดรวม 2,255 RPS ตลอด 20 วินาที แสดงถึงเสถียรภาพของเส้นทางสาธารณะในสถานะทดสอบ
- ค่า latency median (p50) อยู่ราว 15.9–16.0 ms และ p95 ประมาณ 28.5 ms ซึ่งเหมาะสมสำหรับ API แบบสาธารณะ
- หากต้องการ throughput สูงขึ้น ควรพิจารณาเพิ่มการทำงานแบบ asynchronous ภายใน handler, ใช้การทำงานแบบ connection reuse ระหว่างบริการปลายทาง, และตรวจสอบการจับตัวรับส่ง (listener) ของ Bun/Reverse Proxy

- เพิ่มการติดตามตัวชี้วัด (APM/Tracing) เช่น อัตราตอบกลับ, รหัสสถานะ, และ breakdown ของ latency เพื่อระบุคอขวดเมื่อเพิ่มโหลดในอนาคต

4.6.3 ผลการทดสอบประสิทธิภาพบริการจัดการเครื่องเสมือน (Yuzu)

เพื่อประเมินสมรรถนะของบริการ Yuzu ได้ทำการทดสอบการโคลนเครื่องเสมือน (VM Clone) เปรียบเทียบระหว่างวิธี *Linked Clone* และ *Full Clone* โดยใช้แม่แบบ VM เดียวกันและกระจายงานไปยังโหนดปลายทางหลายเครื่อง

ภาพรวมการทดสอบประกอบด้วยทั้งหมด 20 เคส โดยเป็น *Linked Clone* 10 เคส และ *Full Clone* 10 เคส สถานะสำเร็จ/ล้มเหลวและเวลาที่ใช้สรุปได้ดังตารางที่ 4-5 และข้อสังเกตที่ตามมา

ตารางที่ 4-5 สรุปผลการทดสอบการโคลน VM: *Linked* vs *Full*

วิธีโคลน	จำนวนนับ	สำเร็จ	ล้มเหลว	เวลาเฉลี่ย (ms)	ช่วงเวลา (min/max ms)
Linked	10	10	0	5,931.49	1,492.12 / 10,423.62
Full	10	3	7	172,493.80 ¹	167,395.68 / 175,525.81

เมื่อเปรียบเทียบเวลาเฉลี่ย วิธี *Linked Clone* เร็วกว่าวิธี *Full Clone* โดยเฉลี่ยประมาณ 166,562.31 ms หรือคิดเป็นร้อยละ 96.56 ของเวลาที่ลดลง ซึ่งสอดคล้องกับสถาปัตยกรรมการใช้งาน *copy-on-write* ของ *Linked Clone* ที่ไม่คัดลอกดิสก์เต็มทั้งลูกในขั้นตอนเริ่มต้น

ข้อสังเกตและการวิเคราะห์

- อัตราความสำเร็จ: *Linked Clone* สำเร็จ 100% (10/10) ในขณะที่ *Full Clone* สำเร็จ 30% (3/10)
- สาเหตุความล้มเหลวของ *Full Clone*: พบข้อความผิดพลาด เช่น "cfs-lock 'storage-rbd' error: got lock request timeout" และ "failed with exit status: WARNINGS: 1" ซึ่งบ่งชี้ถึงปัญหาการล็อกทรัพยากรของระบบจัดเก็บข้อมูลแบบ RBD/คลัสเตอร์ หรือข้อจำกัดด้านโควตา/นโยบายของสตอเรจ
- ความแปรปรวนของเวลา: เคส *Linked Clone* มีช่วงเวลาดังแต่ 1.49 วินาที ถึง 10.42 วินาที ขึ้นอยู่กับโหนดปลายทางและภาระงาน ณ ขณะนั้น ขณะที่ *Full Clone* ที่สำเร็จใช้เวลาประมาณ 167–175 วินาที (2.8–2.9 นาที) ต่อเคส
- ผลกระทบต่อการออกแบบระบบ: สำหรับงานสอนที่ต้องสร้าง VM จำนวนนักศึกษามากในเวลากำหนด *Linked Clone* ให้เวลาโปรวิชันรวดเร็วอย่างมีนัยสำคัญ และลดความเสี่ยงจากคอขวดด้าน IO ของสตอเรจส่วนกลาง

¹ เวลาเฉลี่ยของ *Full Clone* คำนวณจากเคสที่สำเร็จ (3 เคส) ตามข้อมูลในไฟล์ผลการทดสอบ

ข้อเสนอแนะเชิงปฏิบัติ

- ใช้ Linked Clone เป็นค่าเริ่มต้นในการโปรวิชัน VM จำนวนมาก โดยเฉพาะกรณีแล็บการสอนหรือกิจกรรมที่เน้นความรวดเร็วในการเริ่มใช้งาน
- สำหรับ Full Clone ให้พิจารณาโหมดคิวงาน/การจัดคิวแบบจำกัดพร้อมกัน (throttling) และปรับ *timeout* ของการล็อกสตอเรจ เพื่อหลีกเลี่ยง *cfs-lock timeout*
- ตรวจสอบ คอนฟิก สตอเรจ RBD/ คลัสเตอร์ (เช่น พารามิเตอร์ Ceph/RBD, คิวงานบน Proxmox VE, และเครือข่ายสตอเรจ) เพื่อบรรเทาข้อความผิดพลาดประเภท WARNINGS และเพิ่มอัตราสำเร็จของ Full Clone
- เฝ้าติดตามตัวชี้วัด IO/ล็อกของสตอเรจระหว่างช่วงพัก และทดสอบซ้ำหลังปรับแต่ง เพื่อยืนยันการปรับปรุง

บทที่ 5

สรุปผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

โครงการ "Cloud-Based Platform for Supporting Teaching and Academic Activities" ได้พัฒนาแพลตฟอร์มคลาวด์สำหรับสนับสนุนการเรียนการสอนและกิจกรรมทางวิชาการในภาควิชาเทคโนโลยีสารสนเทศ โดยใช้สถาปัตยกรรมแบบ Microservices ที่ประกอบด้วย 3 ส่วนหลัก ได้แก่ Midori (Frontend), Momoi (API Service), และ Yuzu (VM Operations) ผลลัพธ์ที่สำคัญของโครงการคือการสร้างระบบจัดการเครื่องเสมือน (Virtual Machine) ที่สมบูรณ์ ซึ่งสามารถสร้าง กำหนดค่า เริ่มต้น หยุด และลบเครื่องเสมือนได้ตามความต้องการ พร้อมระบบควบคุมการเข้าถึงแบบ Role-Based Access Control (RBAC) สำหรับนักศึกษา อาจารย์ และผู้ดูแลระบบ ทำให้สามารถใช้ทรัพยากรเซิร์ฟเวอร์ภายในภาควิชาได้อย่างมีประสิทธิภาพ และช่วยเพิ่มความสะดวกในการจัดการสภาพแวดล้อมการเรียนรู้แบบดิจิทัลให้กับอาจารย์และนักศึกษา

5.2 ปัญหาและอุปสรรค

- 5.2.1 ความซับซ้อนของการจัดการ Microservices Architecture เนื่องจากระบบประกอบด้วยหลายส่วนที่ต้องทำงานร่วมกันผ่าน API และ Message Queue ทำให้การดีบั๊กและแก้ไขปัญหาเป็นเรื่องที่ท้าทาย
- 5.2.2 การจัดการ Virtual Machine ผ่าน Proxmox VE API ที่มีความซับซ้อนในการสื่อสารและจัดการ State ของ VM รวมถึงการจัดสรร IP Address และ Resource อย่างอัตโนมัติ
- 5.2.3 การออกแบบระบบ Role-Based Access Control (RBAC) ที่ปลอดภัยและครอบคลุมผู้ใช้งานทุกระดับ ตั้งแต่ นักศึกษา อาจารย์ และผู้ดูแลระบบ โดยต้องคำนึงถึงความปลอดภัยและการจำกัดสิทธิ์การเข้าถึง
- 5.2.4 การใช้เทคโนโลยีใหม่ เช่น Bun Runtime, Elysia.js Framework และ RabbitMQ Message Queue ที่ต้องใช้เวลาในการศึกษาและทำความเข้าใจการทำงาน
- 5.2.5 การจัดการ Monitoring และ Observability ด้วย OpenTelemetry, Jaeger, Prometheus และ Grafana เพื่อติดตามประสิทธิภาพของระบบ

5.3 การแก้ปัญหา

- 5.3.1 ใช้ Docker และ Docker Compose ในการจัดการ Microservices เพื่อให้แต่ละ Service สามารถทำงานแยกกันได้อย่างเป็นอิสระ และจัดทำ API Documentation ด้วย OpenAPI เพื่อให้การสื่อสารระหว่าง Service เป็นมาตรฐาน
- 5.3.2 พัฒนา Yuzu Service ที่ทำหน้าที่เป็น Wrapper สำหรับ Proxmox VE API โดยใช้ RabbitMQ เป็น Message Queue เพื่อจัดการ VM Operations แบบ Asynchronous และสร้างระบบจัดการ IP Pool อัตโนมัติ
- 5.3.3 ออกแบบ Database Schema ด้วย Prisma ORM เพื่อจัดการข้อมูลผู้ใช้และสิทธิ์การเข้าถึง และใช้ JWT Token สำหรับ Authentication พร้อมกำหนด Role และ Permission ที่ชัดเจน
- 5.3.4 จัดทำ Documentation และ Example Code ที่ครอบคลุม พร้อมใช้ TypeScript เป็นหลักในการพัฒนาเพื่อให้ได้ Type Safety และลดข้อผิดพลาดในการพัฒนา
- 5.3.5 ใช้ OpenTelemetry เป็น Standard สำหรับ Observability และติดตั้ง Monitoring Stack ประกอบด้วย Jaeger สำหรับ Distributed Tracing, Prometheus สำหรับ Metrics Collection และ Grafana สำหรับ Visualization

5.4 ข้อเสนอแนะ

- 5.4.1 พัฒนาระบบ Auto-scaling และ Load Balancing เพื่อรองรับจำนวนผู้ใช้งานที่เพิ่มขึ้น และเพิ่มความสามารถในการจัดการ Resource Allocation ให้มีประสิทธิภาพมากขึ้น
- 5.4.2 เพิ่มฟีเจอร์ VM Template Management เพื่อให้อาจารย์สามารถสร้างและจัดการ Template สำหรับรายวิชาต่างๆ ได้ตามความต้องการ และเพิ่มระบบ Backup และ Snapshot Management
- 5.4.3 พัฒนา Mobile Application หรือ Progressive Web App (PWA) เพื่อให้สามารถเข้าถึงระบบได้จากอุปกรณ์มือถือ และเพิ่มระบบ Notification เพื่อแจ้งเตือนสถานะของ VM
- 5.4.4 เพิ่มระบบ CI/CD Pipeline สำหรับการ Deploy และ Testing อัตโนมัติ และจัดทำ Comprehensive Testing Suite ประกอบด้วย Unit Tests, Integration Tests และ End-to-End Tests
- 5.4.5 ศึกษาการใช้ Kubernetes สำหรับการจัดการ Container Orchestration เพื่อเพิ่มความยืดหยุ่นและ Scalability

บรรณานุกรม

- Terry Anderson. (2008). The theory and practice of online learning (2nd).
- Jonathan ChyaXunfei Jiang. (2024). Building a cloud infrastructure for virtual machine scheduling in datacenters [Accessed 25 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 25, 2025, from <https://ieeexplore.ieee.org/document/10427797>
- Richard C. ClarkAnn Kwinn. (2016). The new virtual classroom: Evidence-based guidelines for synchronous e-learning (2nd).
- Carlos GarciaMaria Rodriguez. (2023). Performance evaluation of hypervisor technologies in educational environments: Vmware vsphere vs. proxmox ve vs. kvm. *Journal of Network and Computer Applications*, 203, 103–121. <https://doi.org/10.1016/j.jnca.2022.103401>
- Shahidullah KaiserMd. SadunhaqAli Aman TosunTurgay Korkmaz. (2022). Container technologies for arm architecture: A comprehensive survey of the state-of-the-art [Accessed 23 June 2025.]. *IEEE Communications Surveys Tutorials*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/9852232>
- Paul Hoffman Karen Scarfone. (2011). *Guide to security for full virtualization technologies*. Retrieved June 23, 2025, from <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-125.pdf>
- David LeeWei Chen. (2021). Container technology in software development education: A comparative study of docker and kubernetes. *IEEE Transactions on Education*, 64(2), 187–194. <https://doi.org/10.1109/TE.2020.3015678>
- Naresh NayakDennis GreweSebastian Schildt. (2023). Automotive container orchestration: Requirements, challenges and open directions [Accessed 23 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/10136278>
- Rawiporn SuamsiriKamonwan Janmanee. (2024). Progress tracking system for special project documents [Accessed 23 June 2025.]. Retrieved June 23, 2025, from <http://202.44.47.109/project2/2567-1/671015-1.pdf>

บรรณานุกรม (ต่อ)

- Danairat T. (2015). *Cloud computing technology the architecture overview*. Retrieved June 23, 2025, from https://www.dga.or.th/wp-content/uploads/2015/05/file_c08274b0a8d4c29561b7e7314afaa102.pdf
- Yu TaoGang Chen. (2016). An extensible universal reverse proxy architecture [Accessed 23 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/7945939>

ภาคผนวก

ภาคผนวก ก.

การติดตั้งและใช้งาน Cloud-Based Platform

ก.1 การติดตั้งและใช้งาน Cloud-Based Platform

ในการติดตั้งและใช้งาน Cloud-Based Platform สำหรับการสนับสนุนกิจกรรมการเรียนรู้การสอนและกิจกรรมทางวิชาการ ผู้ใช้งานจำเป็นต้องเตรียมความพร้อมของระบบและทำการติดตั้งตามขั้นตอนดังนี้

ข้อกำหนดของระบบ (System Requirements)

- 1) ซอฟต์แวร์ที่จำเป็น:
 - 1.1) Bun 1.2.0 หรือสูงกว่า (<https://bun.sh/>)
 - 1.2) Docker และ Docker Compose (<https://www.docker.com/>)
 - 1.3) PostgreSQL 15 หรือสูงกว่า
 - 1.4) RabbitMQ 3.12 หรือสูงกว่า
 - 1.5) Proxmox VE (สำหรับการใช้งานจริง)
- 2) ข้อกำหนดของฮาร์ดแวร์:
 - 2.1) RAM: อย่างน้อย 8 GB (แนะนำ 16 GB หรือมากกว่า)
 - 2.2) CPU: อย่างน้อย 4 cores
 - 2.3) พื้นที่เก็บข้อมูล: อย่างน้อย 50 GB
 - 2.4) เชื่อมต่ออินเทอร์เน็ตเสถียร

การดาวน์โหลดและติดตั้งโปรเจกต์

- 1) เปิด Terminal หรือ Command Prompt และทำการโคลนโปรเจกต์จาก GitHub Repository โดยใช้คำสั่งดังภาพที่ ก-1

```
git clone https://github.com/akikungz/cloud-based-platform.git
cd cloud-based-platform
```

ภาพที่ ก-1 คำสั่งสำหรับโคลนโปรเจกต์

- 2) ติดตั้ง Dependencies ของโปรเจกต์ โดยใช้คำสั่งดังภาพที่ ก-2

```
bun install
```

ภาพที่ ก-2 คำสั่งสำหรับติดตั้ง Dependencies

- 3) ตั้งค่า Environment Variables โดยคัดลอกไฟล์ตัวอย่าง โดยใช้คำสั่งดังภาพที่ ก-3

ก.2 การเริ่มต้นใช้งานระบบ

หลังจากติดตั้งโปรเจกต์เรียบร้อยแล้ว สามารถเริ่มต้นใช้งานระบบได้ดังนี้

```
cp .env.example .env
```

ภาพที่ ก-3 การตั้งค่า Environment Variables

การรันระบบในโหมด Development

- 1) เริ่มต้น Infrastructure Services ด้วย Docker Compose โดยใช้คำสั่งดังภาพที่ ก-4

```
docker-compose up -d
```

ภาพที่ ก-4 การเริ่มต้น Infrastructure Services

- 2) เริ่มต้น Midori (Frontend Application) โดยใช้คำสั่งดังภาพที่ ก-5

```
cd apps/midori
bun dev
```

ภาพที่ ก-5 การเริ่มต้น Midori Frontend

- 3) เริ่มต้น Momoi (API Service) ใน Terminal ใหม่ โดยใช้คำสั่งดังภาพที่ ก-6
- 4) เริ่มต้น Yuzu (VM Operations Service) ใน Terminal ใหม่ โดยใช้คำสั่งดังภาพที่ ก-7
- 5) เข้าใช้งานระบบผ่าน Web Browser ที่ <http://localhost:3000>

ก.3 การใช้งานส่วนประกอบต่างๆ ของระบบ

ระบบ Cloud-Based Platform ประกอบด้วยส่วนประกอบหลัก 3 ส่วน ดังนี้

Midori - Frontend Application

- 1) เป็น Web Application ที่พัฒนาด้วย Next.js 15.5
- 2) ใช้ Material UI 7.3 และ Tailwind CSS 4 สำหรับส่วน User Interface
- 3) รองรับการใช้งานบนอุปกรณ์ Desktop และ Mobile
- 4) มีฟีเจอร์หลัก ดังนี้:
 - 4.1) การจัดการ Virtual Machines (สร้าง, ติดตาม, จัดการ)
 - 4.2) Dashboard สำหรับดูการใช้งานทรัพยากร
 - 4.3) ระบบ Authentication และการจัดการบัญชีผู้ใช้
 - 4.4) รองรับการใช้งานแบบ Responsive Design
- 5) URL สำหรับการเข้าใช้งาน: <http://localhost:3000>

Momoi - API Service

- 1) เป็น Core API Service ที่พัฒนาด้วย Elysia.js

```
cd apps/momoi
bun dev
```

ภาพที่ ก-6 การเริ่มต้น Momoi API Service

```
cd apps/yuzu
bun dev
```

ภาพที่ ก-7 การเริ่มต้น Yuzu VM Operations Service

- 2) ทำหน้าที่เป็นตัวกลางระหว่าง Frontend และ VM Operations Service
- 3) มีฟีเจอร์หลัก ดังนี้:
 - 3.1) RESTful API สำหรับการทำงานของแพลตฟอร์ม
 - 3.2) OpenAPI Documentation แบบอัตโนมัติ
 - 3.3) การติดตาม Telemetry ด้วย Jaeger และ Prometheus
 - 3.4) การเชื่อมต่อกับ RabbitMQ สำหรับการประมวลผลแบบ Asynchronous
- 4) URL สำหรับ API: <http://localhost:3001>

Yuzu - VM Operations Service

- 1) เป็น Service สำหรับจัดการ Virtual Machines ใน Proxmox VE
- 2) ประมวลผลคำขอผ่าน RabbitMQ Message Queue
- 3) มีฟีเจอร์หลัก ดังนี้:
 - 3.1) จัดการ VM Lifecycle (สร้าง, เริ่ม, หยุด, ลบ VMs)
 - 3.2) การ Provisioning แบบ Template-based
 - 3.3) การจัดการ IP Address แบบอัตโนมัติ
 - 3.4) ระบบ Logging ที่ครอบคลุมด้วย Pino

ก.4 การตรวจสอบและติดตามระบบ

ระบบมี Services สำหรับการตรวจสอบและติดตามการทำงานดังนี้

Monitoring Services

- 1) **Prometheus** - การเก็บข้อมูล Metrics
 - 1.1) URL: <http://localhost:9090>
 - 1.2) ใช้สำหรับติดตามประสิทธิภาพของระบบ
- 2) **Jaeger** - การติดตาม Distributed Tracing
 - 2.1) URL: <http://localhost:16686>

- 2.2) ใช้สำหรับติดตามการทำงานของ API calls
- 3) **Grafana** - Dashboard สำหรับการแสดงผล
 - 3.1) URL: <http://localhost:3002>
 - 3.2) Username/Password: admin/admin (ค่าเริ่มต้น)
 - 3.3) ใช้สำหรับแสดงข้อมูลสถิติและ Metrics
- 4) **RabbitMQ Management** - การจัดการ Message Queue
 - 4.1) URL: <http://localhost:15672>
 - 4.2) Username/Password: user/password (ค่าเริ่มต้น)
 - 4.3) ใช้สำหรับตรวจสอบ Message Queue

ก.5 การแก้ไขปัญหาเบื้องต้น

หากพบปัญหาในการใช้งาน สามารถแก้ไขได้ดังนี้

ปัญหาที่พบบ่อยและวิธีแก้ไข

- 1) **Port ถูกใช้งานแล้ว**
 - 1.1) ตรวจสอบ Port ที่ใช้งานด้วยคำสั่ง: `lsof -i :3000` (สำหรับ macOS/Linux)
 - 1.2) หยุดกระบวนการที่ใช้ Port หรือเปลี่ยน Port ในไฟล์ .env
- 2) **Docker Services ไม่สามารถเริ่มได้**
 - 2.1) ตรวจสอบสถานะด้วยคำสั่ง: `docker-compose ps`
 - 2.2) ดู Log ด้วยคำสั่ง: `docker-compose logs [service_name]`
 - 2.3) รีสตาร์ท Services ด้วยคำสั่ง: `docker-compose restart`
- 3) **Database Connection Error**
 - 3.1) ตรวจสอบว่า PostgreSQL Container ทำงานอยู่
 - 3.2) ตรวจสอบการตั้งค่าใน Environment Variables
 - 3.3) รันคำสั่ง Database Migration (หากจำเป็น)
- 4) **Frontend ไม่สามารถเชื่อมต่อ API ได้**
 - 4.1) ตรวจสอบว่า Momoi API Service ทำงานอยู่ที่ Port 3001
 - 4.2) ตรวจสอบการตั้งค่า API URL ในไฟล์ Environment
 - 4.3) ตรวจสอบ CORS Settings ใน API Service

การเข้าถึงข้อมูลเพิ่มเติม

- 1) อ่าน Documentation เพิ่มเติมใน README.md ของแต่ละ Component
- 2) ตรวจสอบ Log Files สำหรับข้อมูลการ Debug
- 3) ใช้ Jaeger UI สำหรับติดตามการทำงานของ API
- 4) ใช้ Grafana Dashboard สำหรับติดตามประสิทธิภาพระบบ